ITCS213 Computer Organization and Architecture

Class: 1-2026
Sec 2 Fri 9:00-12:00 IT312 (Aj. Joke)
Sec 3 Fri 13:00-16:00 IT312 (Aj. Joke)
Sec 1 Fri 13:00-16:00 IT311 (Aj. Pete)

Unit 9: I-Type vs. R-Type Registers Usages

Lecturer Dr. Kanoksak Wattanachote (金功勳 博士)
kanoksak.wat@mahidol.ac.th

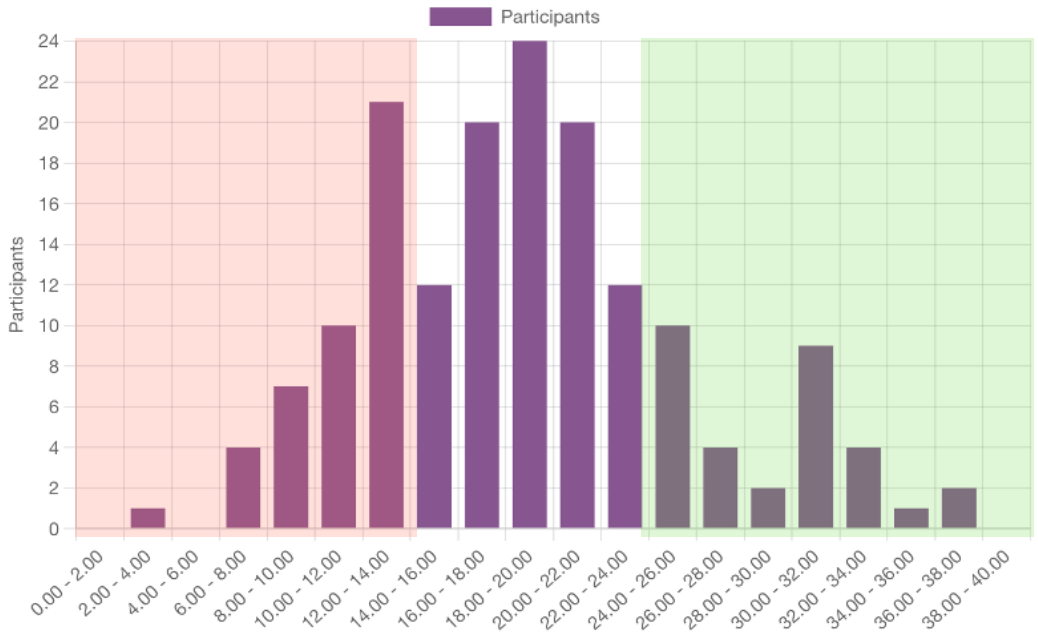
Grading

ITCS213 Computer Organization and Architecture

- Overall Score Parts
  - Class activities + Assignments + Quizzes 20%
  - Midterm Exam 40%
  - Final Exam 40%

GRADING SYSTEM

Grade Criteria	A	>=	80
	B+	>=	75
	B	>=	70
	C+	>=	65
	C	>=	60
	D+	>=	55
	D	>=	50
	F	<	50



Content

Unit 9: I-Type vs. R-Type Registers Usages

- I-Type Arithmetic and Logical Instructions
- Pseudo Instructions
- Integer Multiplication in MIPS

R-Type Instruction Format (Recap)

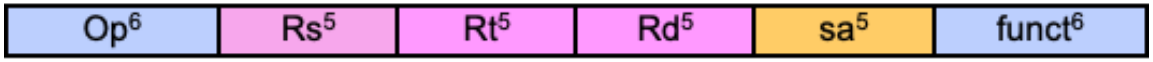
R-Type Instructions

Unit 9: I-Type vs. R-Type Registers Usages

- The key to understanding the instructions is to familiarize yourself with their format.
- All MIPS instructions are 32 bits long, but are divided differently.
- R-Type (Register-Type)
  - Purpose: For operations where all operands are registers, e.g.,

    ```
    add $t1, $t2, $t3
    ```

  - Format:



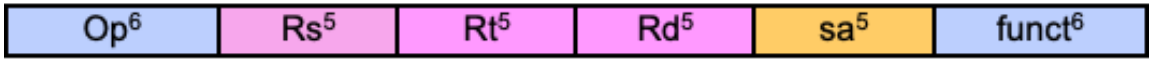
  - Why **`Op`** is often `0`:
    - The **`Op`** field (the first 6 bits) is the operation code.
    - For most basic R-type instructions (**`add`**, **`sub`**, **`and`**, **`or`**, etc.), the **`Op`** code is `000000` — this tells the CPU:
      - "Hey, this is an R-type instruction — to find out which R-type instruction, look at the very last 6 bits, the **`funct`** field."



R-Type Instructions

Unit 9: I-Type vs. R-Type Registers Usages

- The example: `funct` code `100000` means **`add`**, `100100` means **`and`**.
- The other fields are:
  - `Rs`: First source register
  - `Rt`: Second source register
  - `Rd`: Destination register (where the result goes)
  - `sa` or `shamt`: Shift amount (used for shift instructions like **`sll`**, otherwise `0`)
  - **`funct`**: Function code (specifies the exact operation)



Walkthrough of R-Type Datapatl

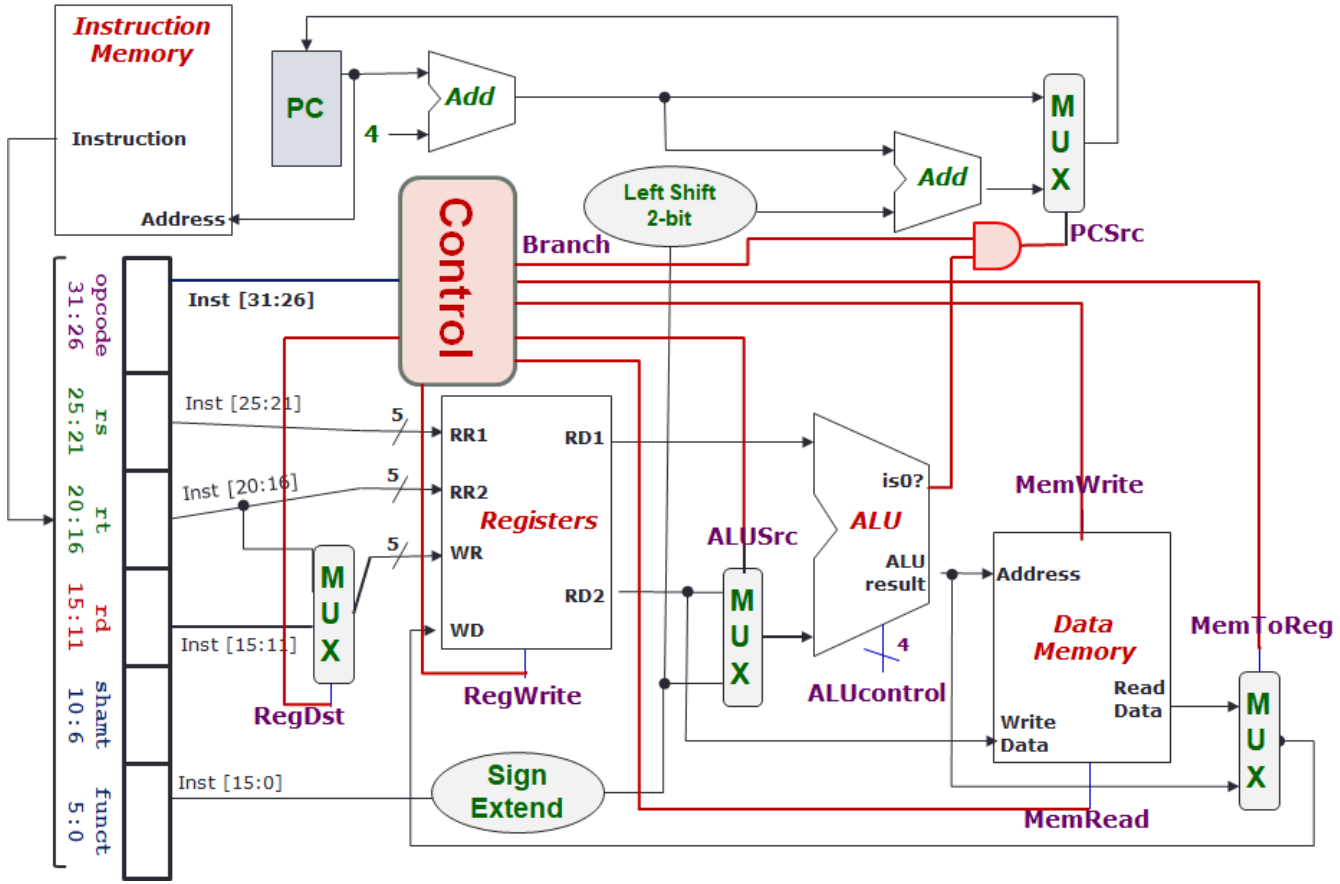
Unit 9: I-Type vs. R-Type Registers Usages

opcode (6)	rs (5)	rt (5)	rd (5)	sa (5)	function (6)

Instruction		Function
add	rd, rs, rt	100000
addu	rd, rs, rt	100001
and	rd, rs, rt	100100

- Let's follow the path of an `add $t0, $t1, $t2` instruction through this diagram.
  - What are they `rs`, `rt`, `rd`?
- `$t1` is register 9 and `$t2` is register 10.
  - I can also say:

```
add $8, $9, $10
```



Walkthrough of R-Type Datapatl

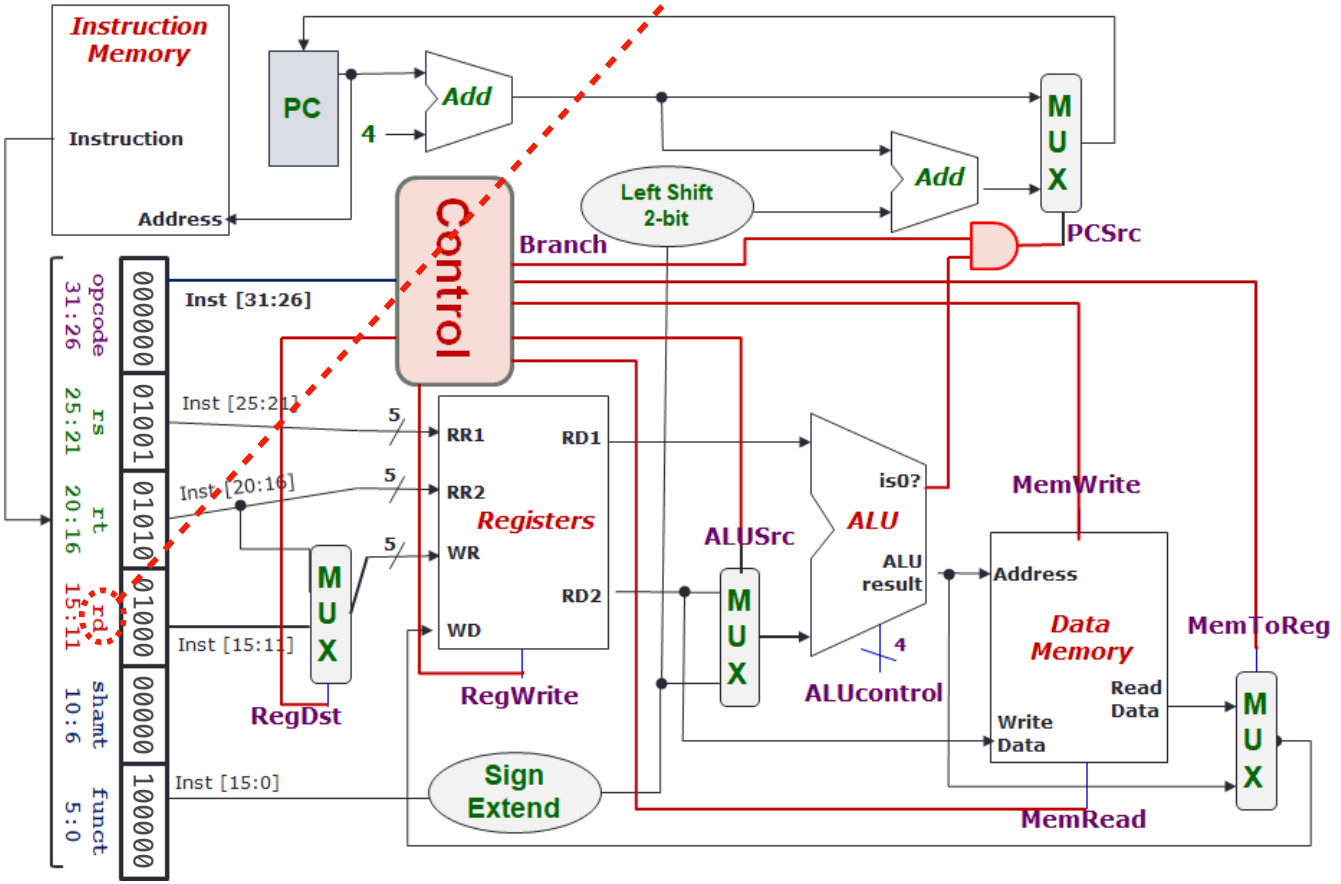
Unit 9: I-Type vs. R-Type Registers Usages

opcode (6)	rs (5)	rt (5)	rd (5)	sa (5)	function (6)

Instruction		Function
add	rd, rs, rt	100000
addu	rd, rs, rt	100001
and	rd, rs, rt	100100

1: Instruction Fetch:

- The Program Counter (PC) holds the address of the current instruction.
- This address is stored in the **Instruction Memory**.
- The instruction memory outputs the 32-bit instruction **word** `00000001001010100100000000100000` (which is the machine code for `add $8, $9, $10`).

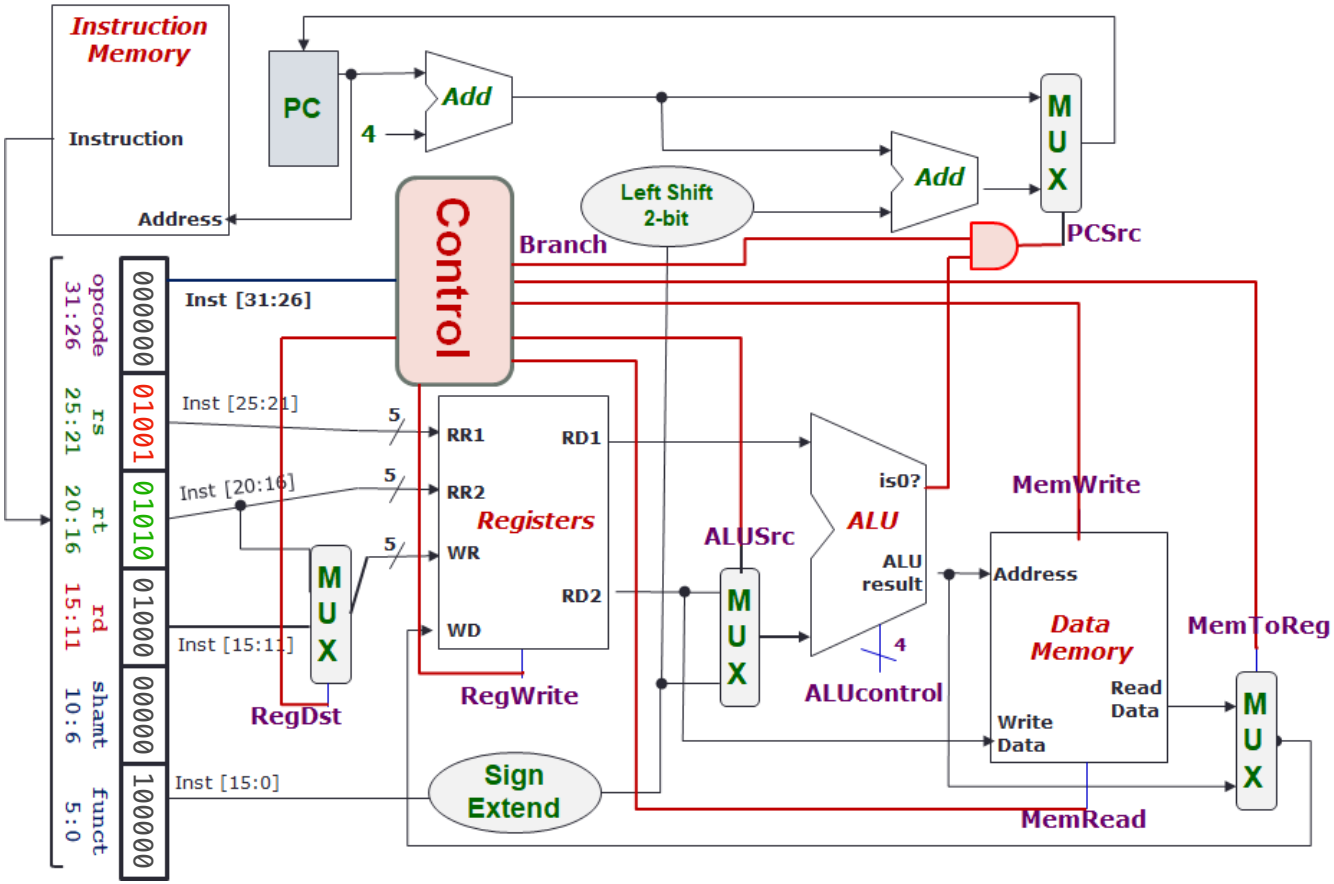


Walkthrough of R-Type Datapath

Unit 9: I-Type vs. R-Type Registers Usages

2: Instruction Decode & Register Fetch:

- The instruction bits are split and sent to different units.
- opcode [31:26] (`000000`) goes to the **Control unit**.
- `rs` [25:21] (`01001` for $t1 or $9) is connected to **Read register 1** (**RR1**) of the register file.
- `rt` [20:16] (`01010` for $t2 or $10) is connected to **Read register 2** (**RR2**) of the register file.

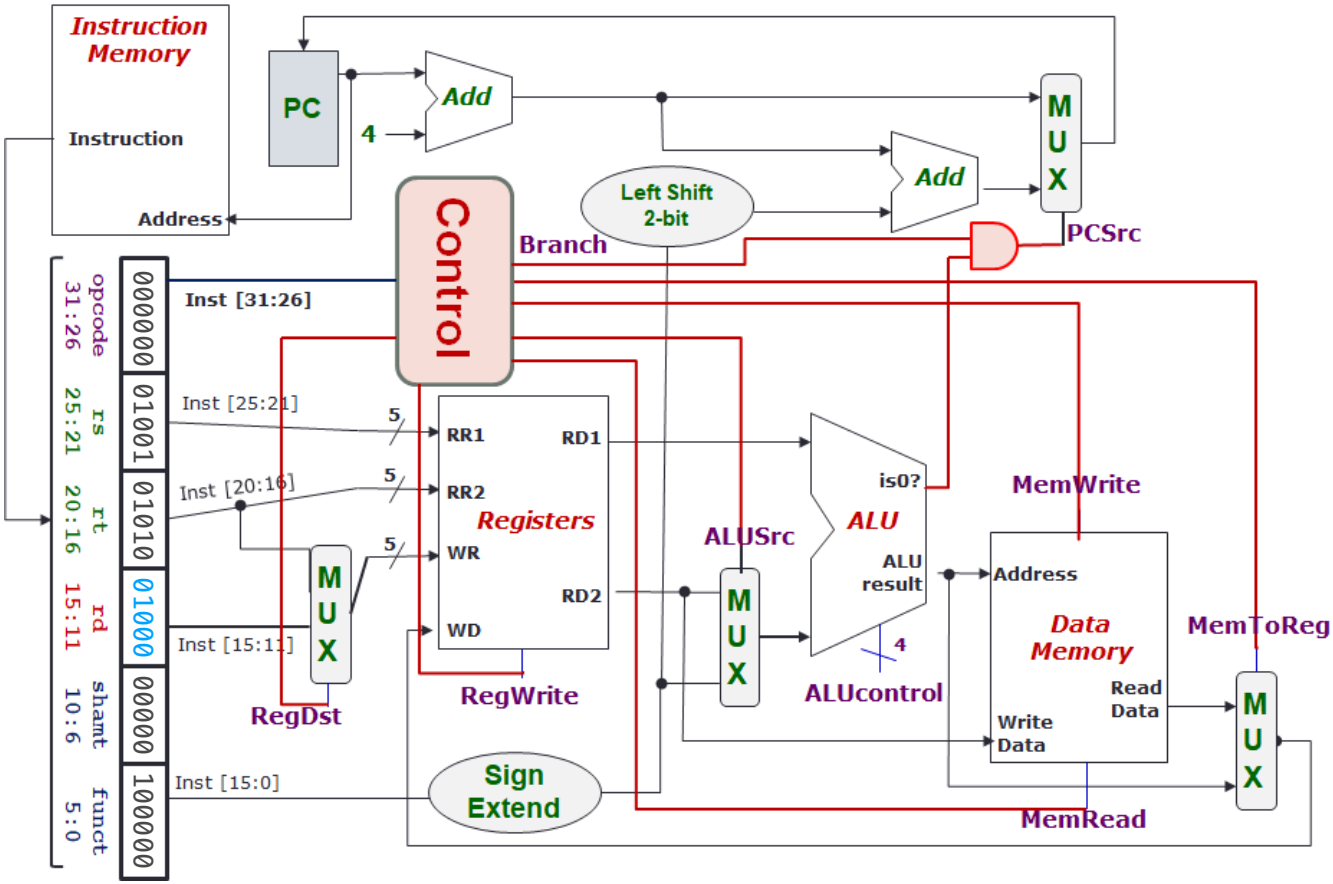


Walkthrough of R-Type Datapath

Unit 9: I-Type vs. R-Type Registers Usages

2: Instruction Decode & Register Fetch:

- `rd` [15:11] (`01000` for $t0 or $8) is connected to the **Write Register** (**WR**) input.
- This is a key difference from **I-type** instructions.

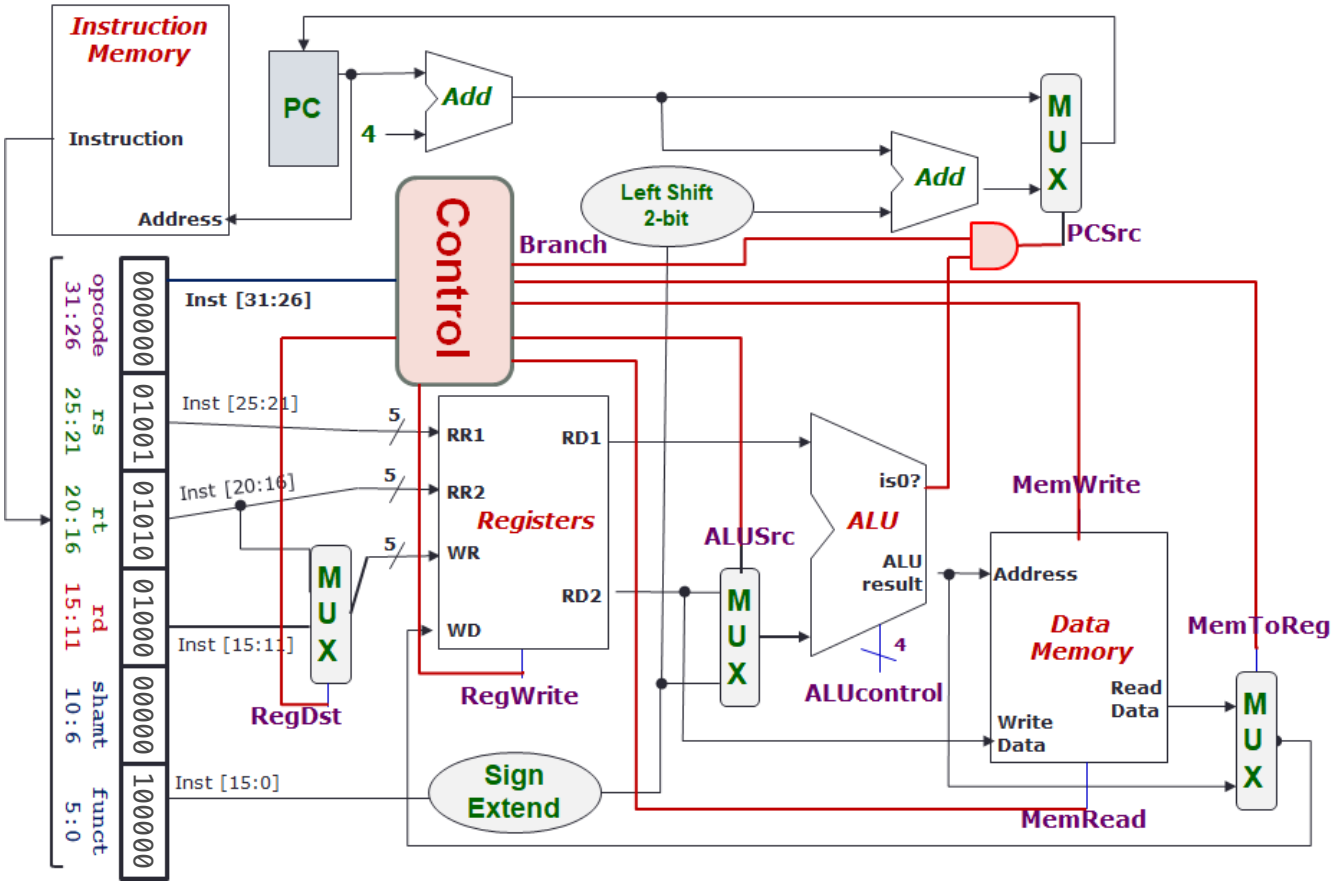


Walkthrough of R-Type Datapath

Unit 9: I-Type vs. R-Type Registers Usages

2: Instruction Decode & Register Fetch:

- The **Control unit** sees the `opcode` is `000000` (**R-type**) and sets its output signals accordingly.

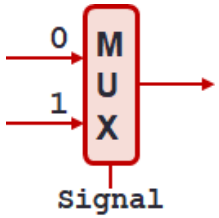


Walkthrough of R-Type Datapatl

Unit 9: I-Type vs. R-Type Registers Usages

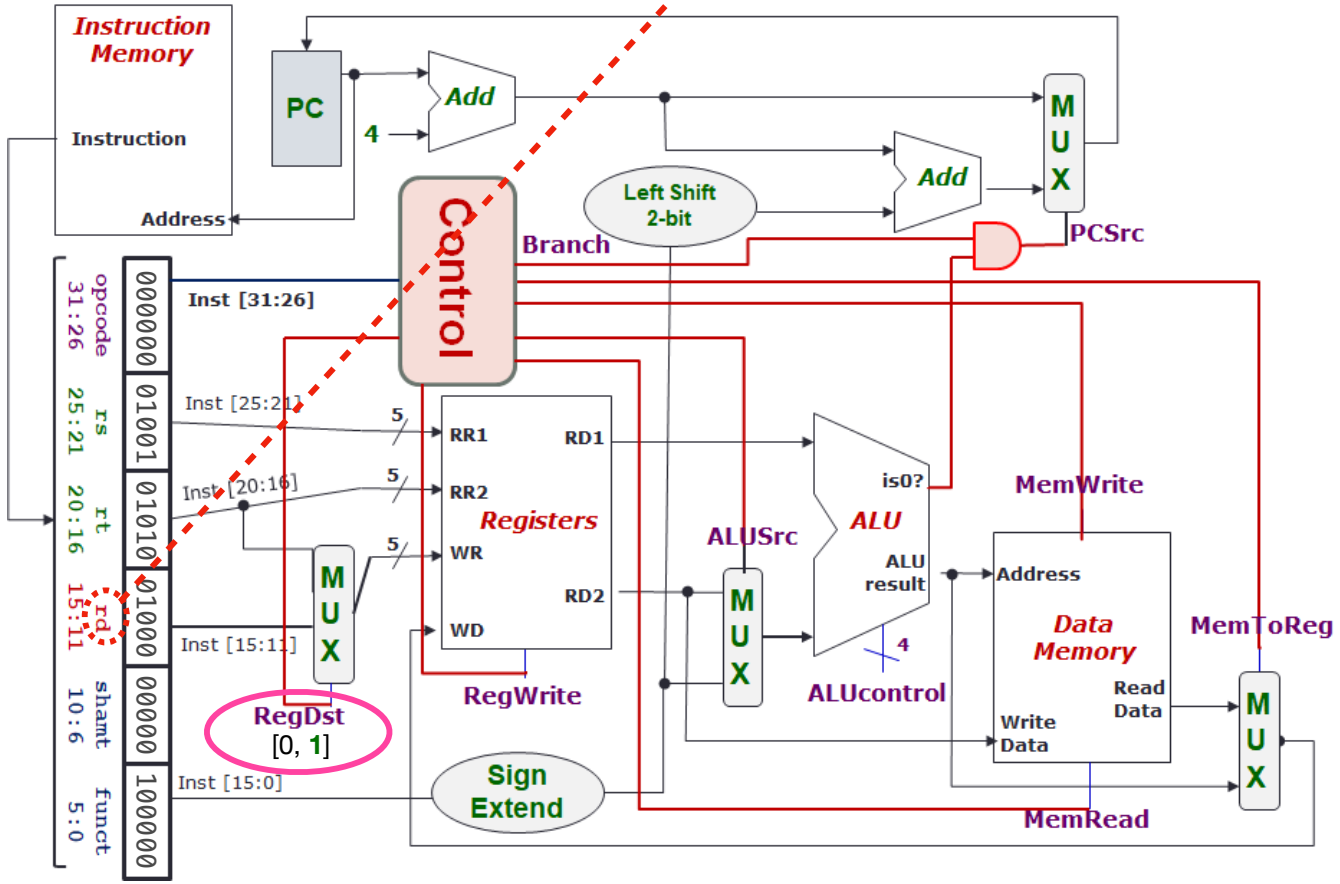
2: Instruction Decode & Register Fetch:

- **RegDst** = 1; this controls a MUX, instructing it to use the `rd` field from the instruction [15:11] as the write register, rather than `rt` [20:16].
  - False (0): Write Register = Inst[20:16] **`rt`**
  - True (1): Write Register = Inst[15:11] **`rd`**



opcode (6)	rs (5)	rt (5)	rd (5)	sa (5)	function (6)

Instruction		Function
add	rd, rs, rt	100000
addu	rd, rs, rt	100001
and	rd, rs, rt	100100

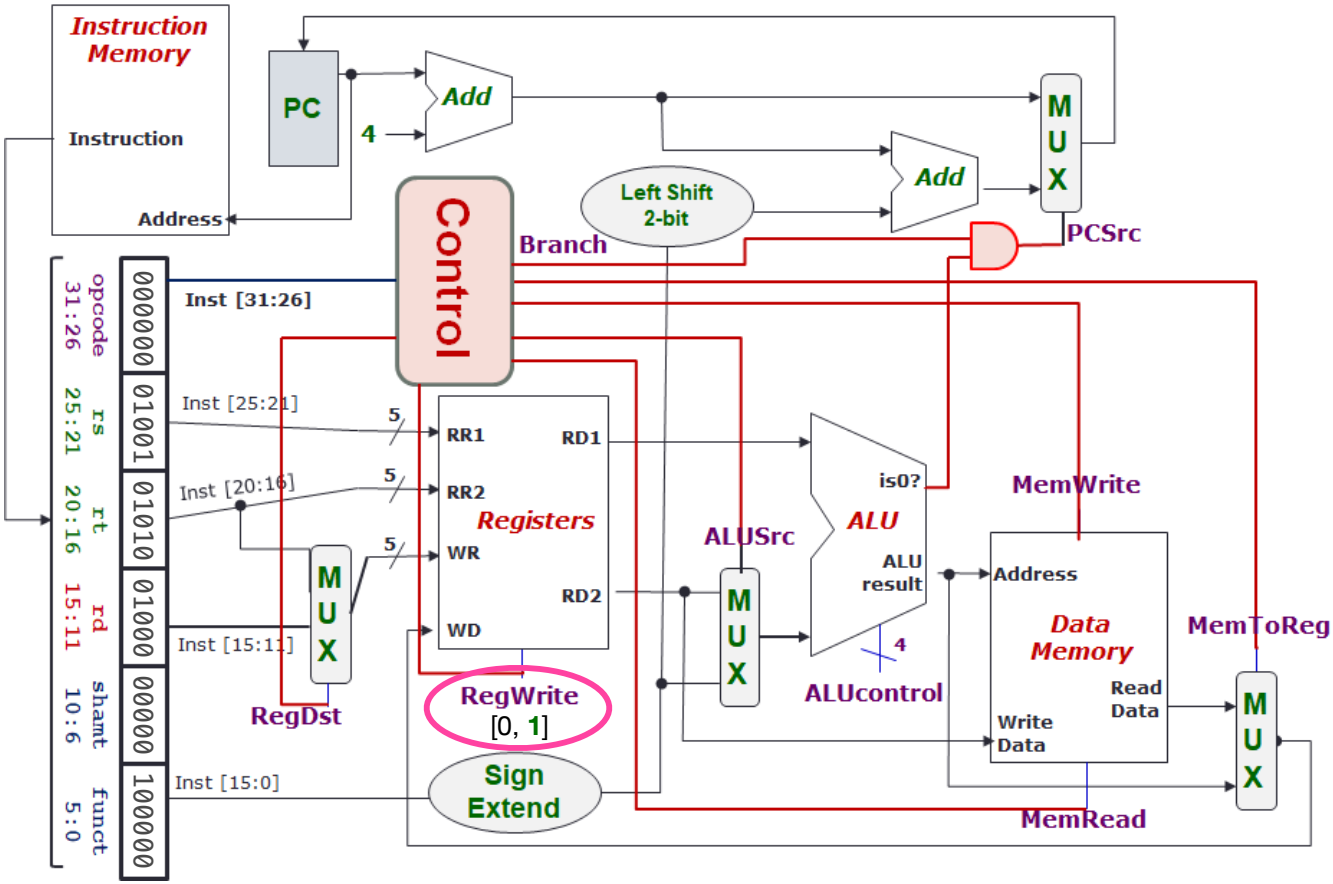


Walkthrough of R-Type Datapath

Unit 9: I-Type vs. R-Type Registers Usages

2: Instruction Decode & Register Fetch:

- **RegWrite** = 1, because we will write to a register at the end.
  - False (0): No register write
  - True (1): New value will be written

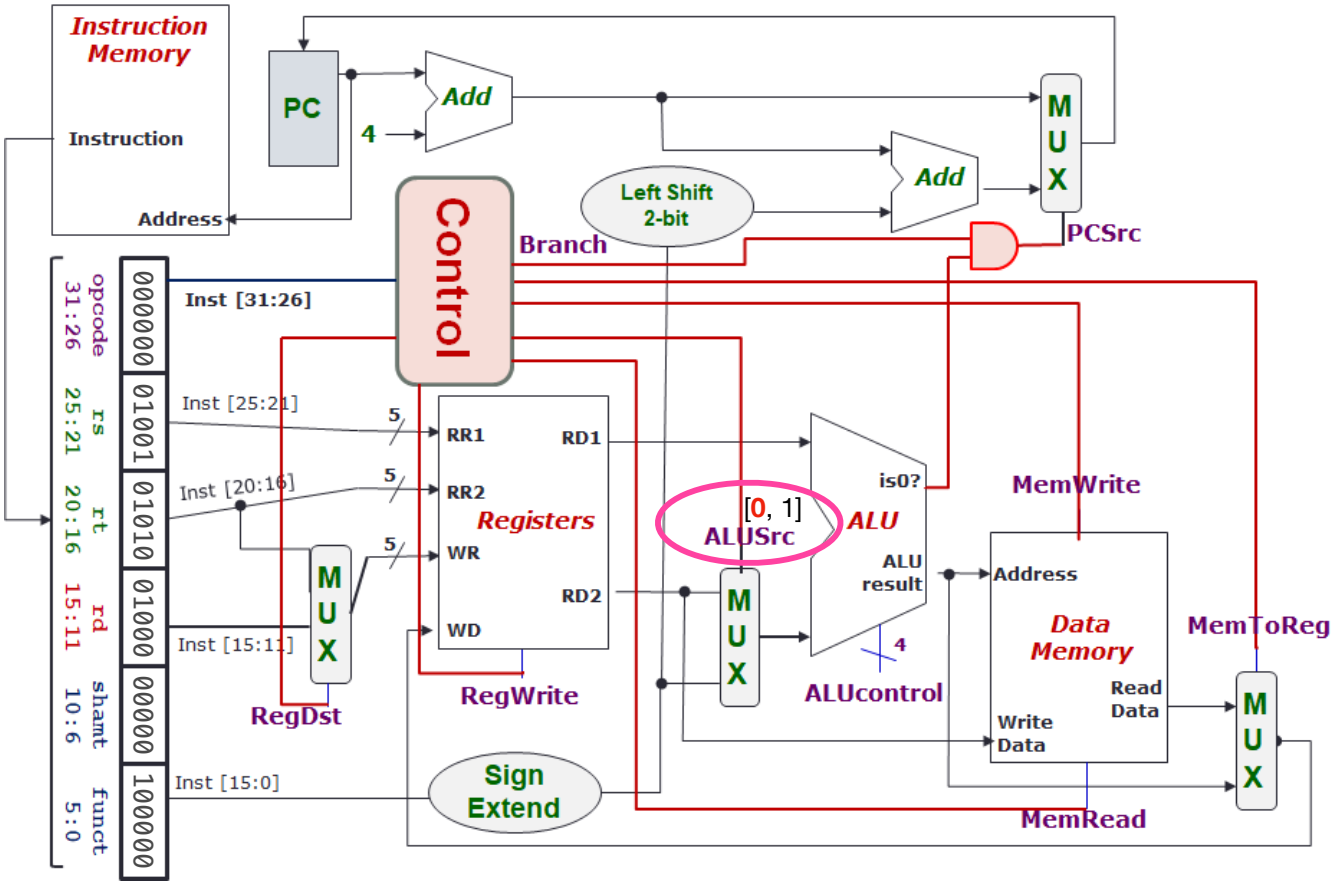


Walkthrough of R-Type Datapath

Unit 9: I-Type vs. R-Type Registers Usages

2: Instruction Decode & Register Fetch:

- **ALUSrc** = 0; this controls a MUX, instructing it to use the second register read from the register file as the second ALU input, rather than the sign-extended immediate value.
  - False (0): ALUop2 = Read Data 2
  - True (1): ALUop2 = sign_extend(Inst[15:0])

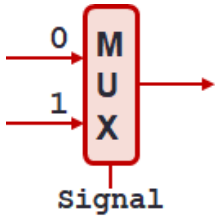


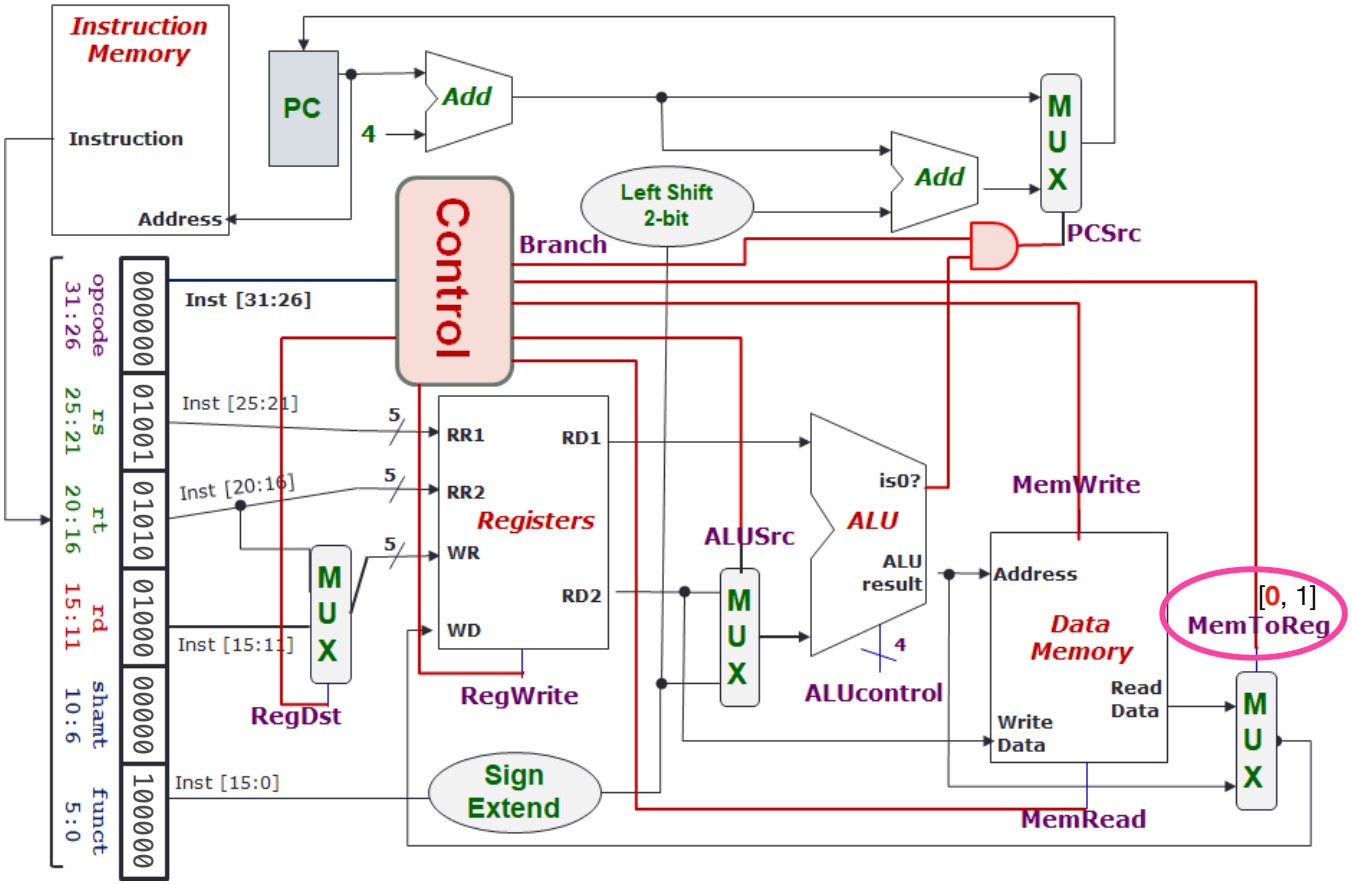
Walkthrough of R-Type Datapath

Unit 9: I-Type vs. R-Type Registers Usages

2: Instruction Decode & Register Fetch:

- **MemToReg** = 0, this controls the final MUX, telling it to write the ALU result back to the register file, not the data from memory.
  - True (1): Register Write Data = Memory Read Data.
  - False (0): Register Write Data = **ALU result**.



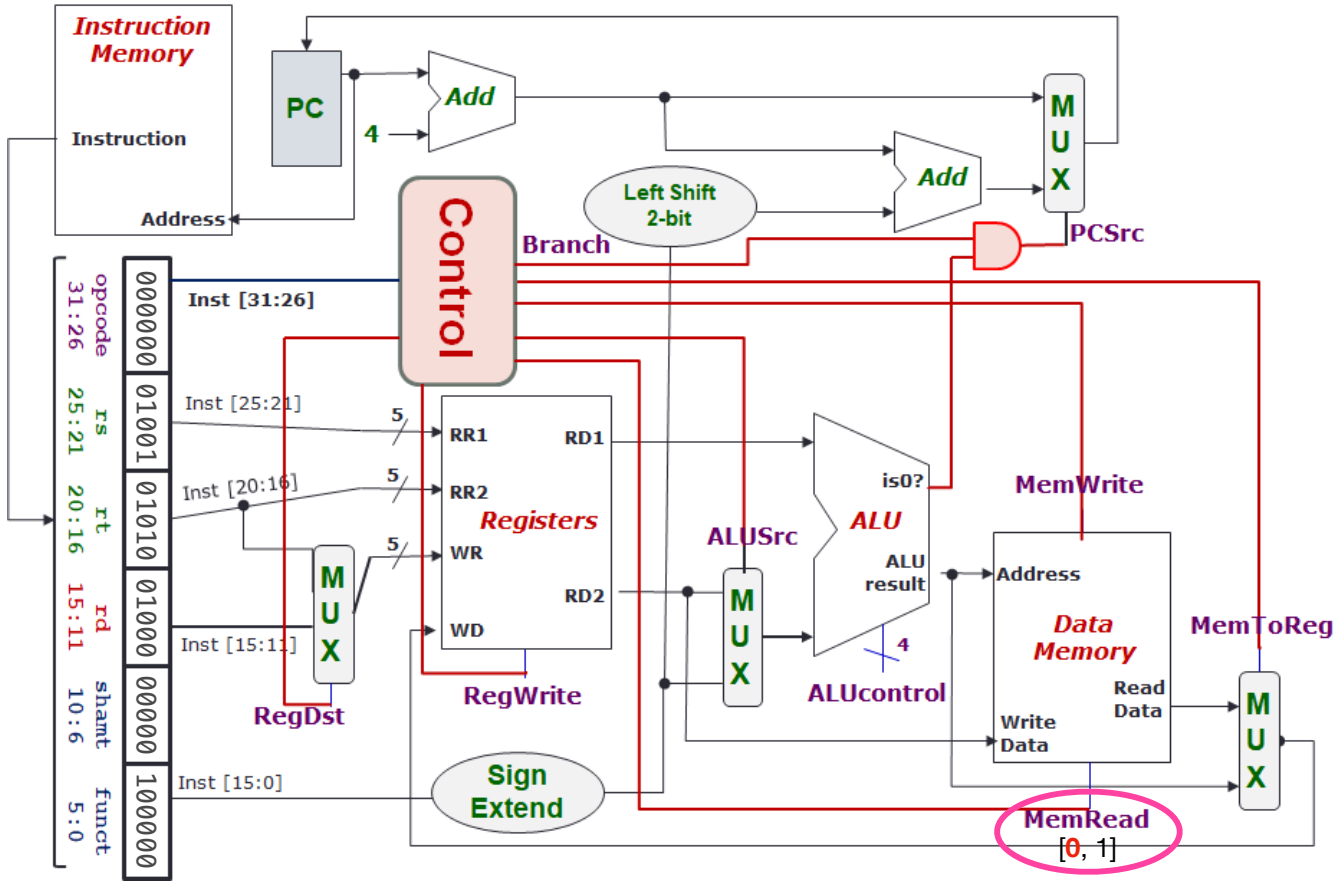


Walkthrough of R-Type Datapath

Unit 9: I-Type vs. R-Type Registers Usages

2: Instruction Decode & Register Fetch:

- **MemRead** = 0
  - False (0): Not performing memory read access.
  - True (1): Read memory using ALU result as **Address**.

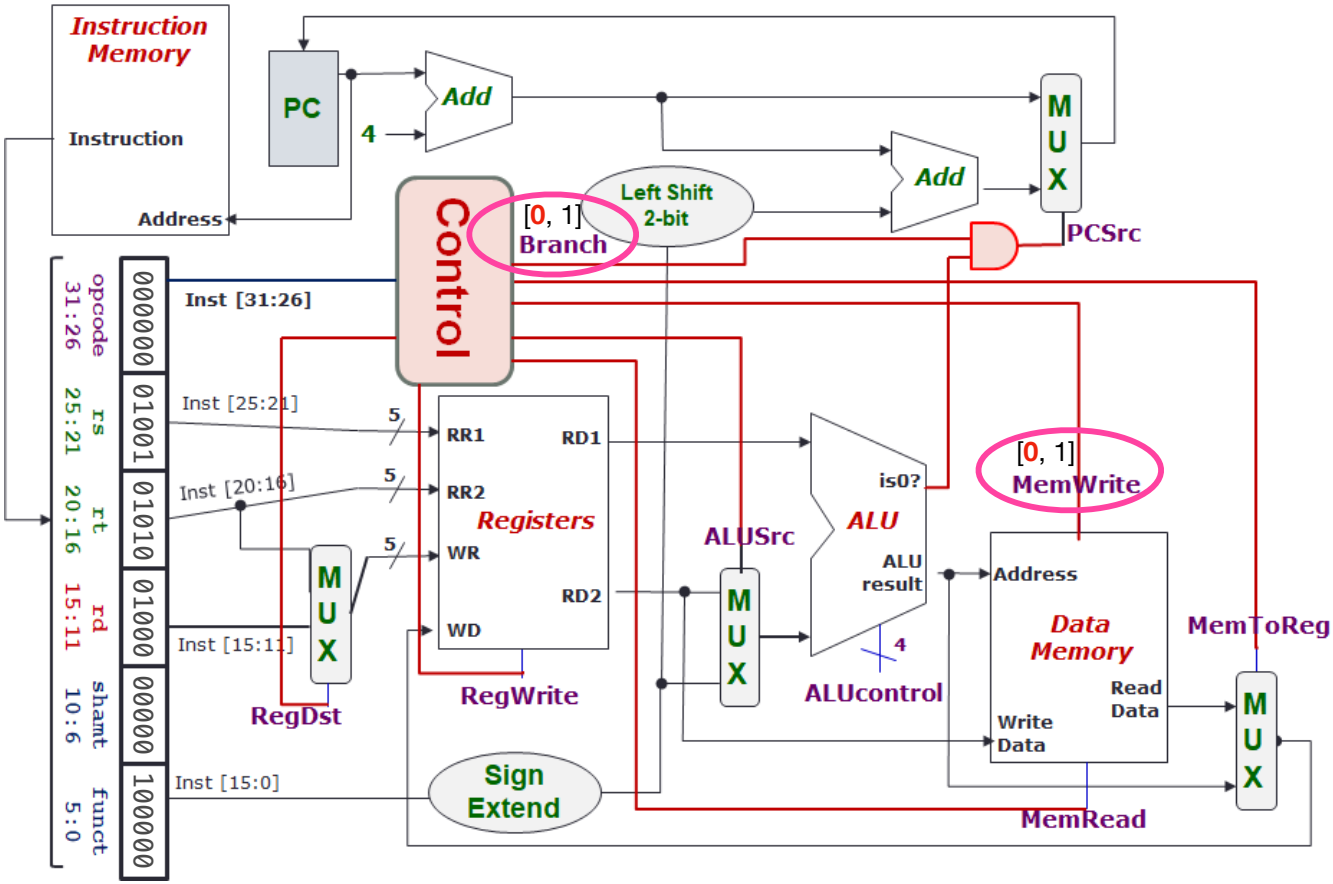


Walkthrough of R-Type Datapath

Unit 9: I-Type vs. R-Type Registers Usages

2: Instruction Decode & Register Fetch:

- **MemWrite** = 0
  - False (0): Not performing memory write operation.
  - True (1): Write to memory at Address (from ALU result) with data from **Read Data 2**.
- Branch = 0

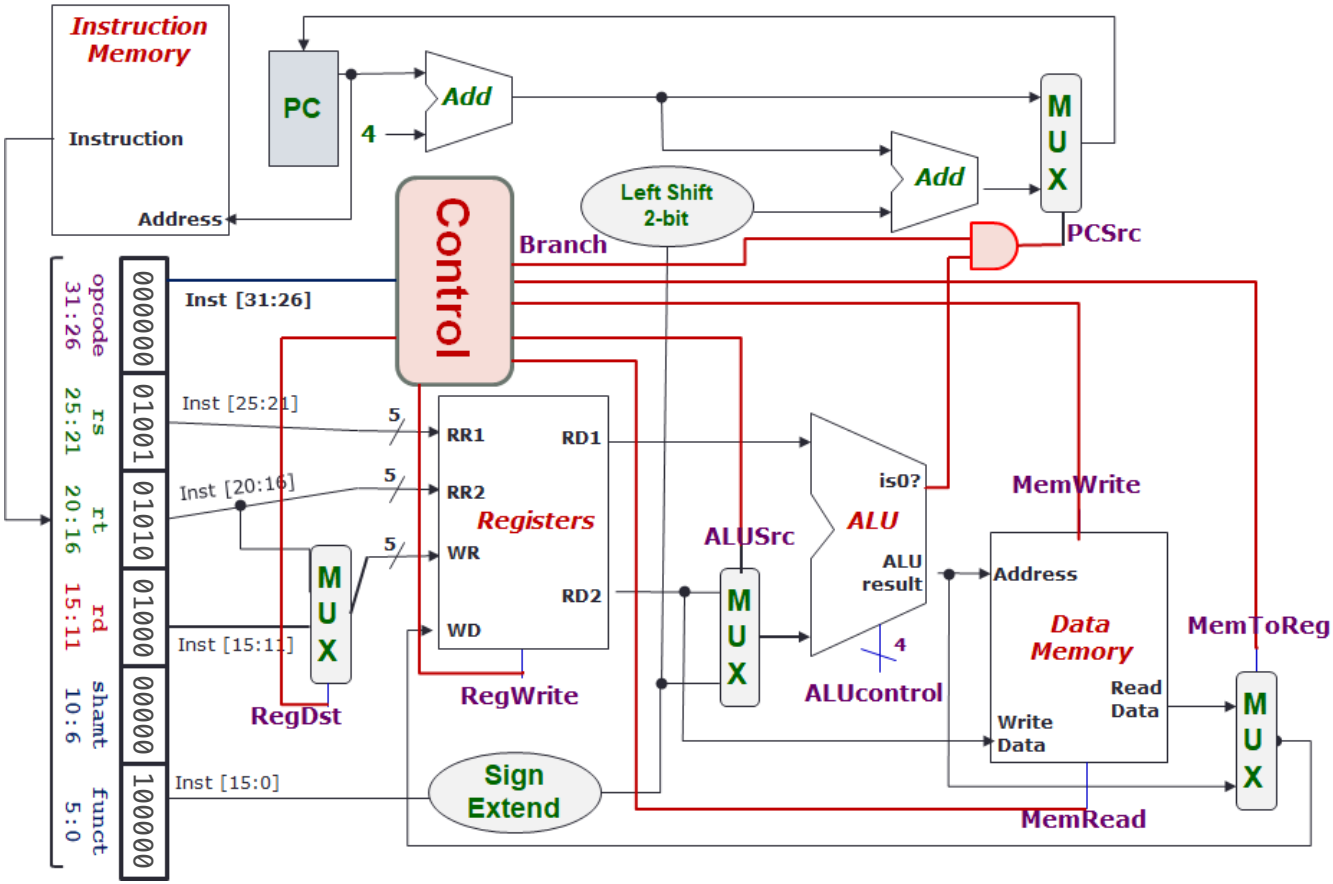


Walkthrough of R-Type Datapath

Unit 9: I-Type vs. R-Type Registers Usages

2: Instruction Decode & Register Fetch:

- The **Register File** reads the values stored in `$t1—rs` and `$t2—rt` and outputs them on Read Data 1 (**RD1**) and Read Data 2 (**RD2**).



Walkthrough of R-Type Datapatl

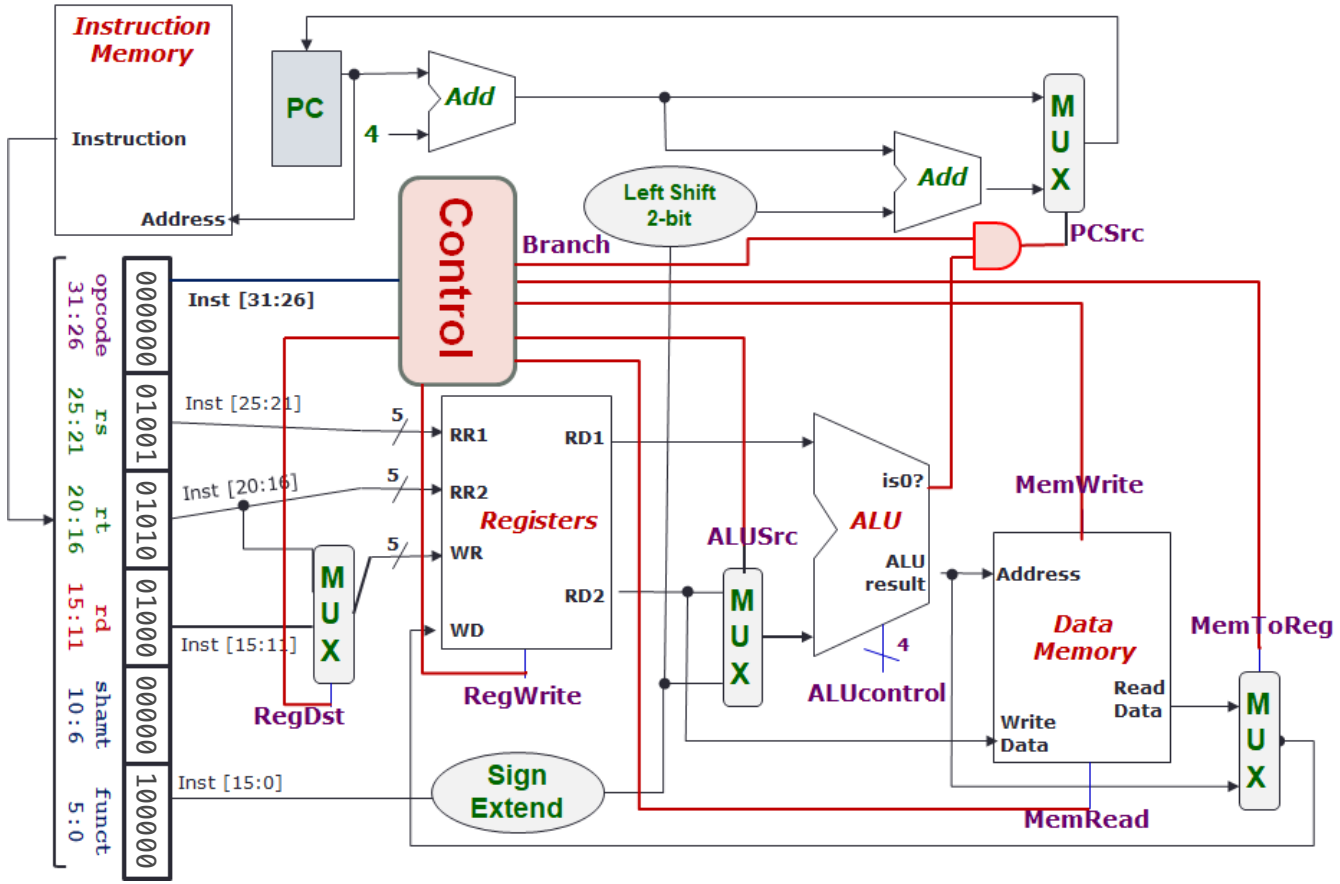
Unit 9: I-Type vs. R-Type Registers Usages

opcode (6)	rs (5)	rt (5)	rd (5)	sa (5)	function (6)

Instruction		Function
add	rd, rs, rt	100000
addu	rd, rs, rt	100001
and	rd, rs, rt	100100

3: Execution:

- **RD1** (value of `$t1`) and **RD2** (value of `$t2`) are sent to the ALU.
- The `funct` field [5:0] (100000 for `add`) is sent to the **ALU Control** unit.
- This unit tells the main ALU to perform an **addition** operation (ALUop = `add`).
- The ALU performs the **addition** on the two values from the **Register File** and outputs the **ALU result**.

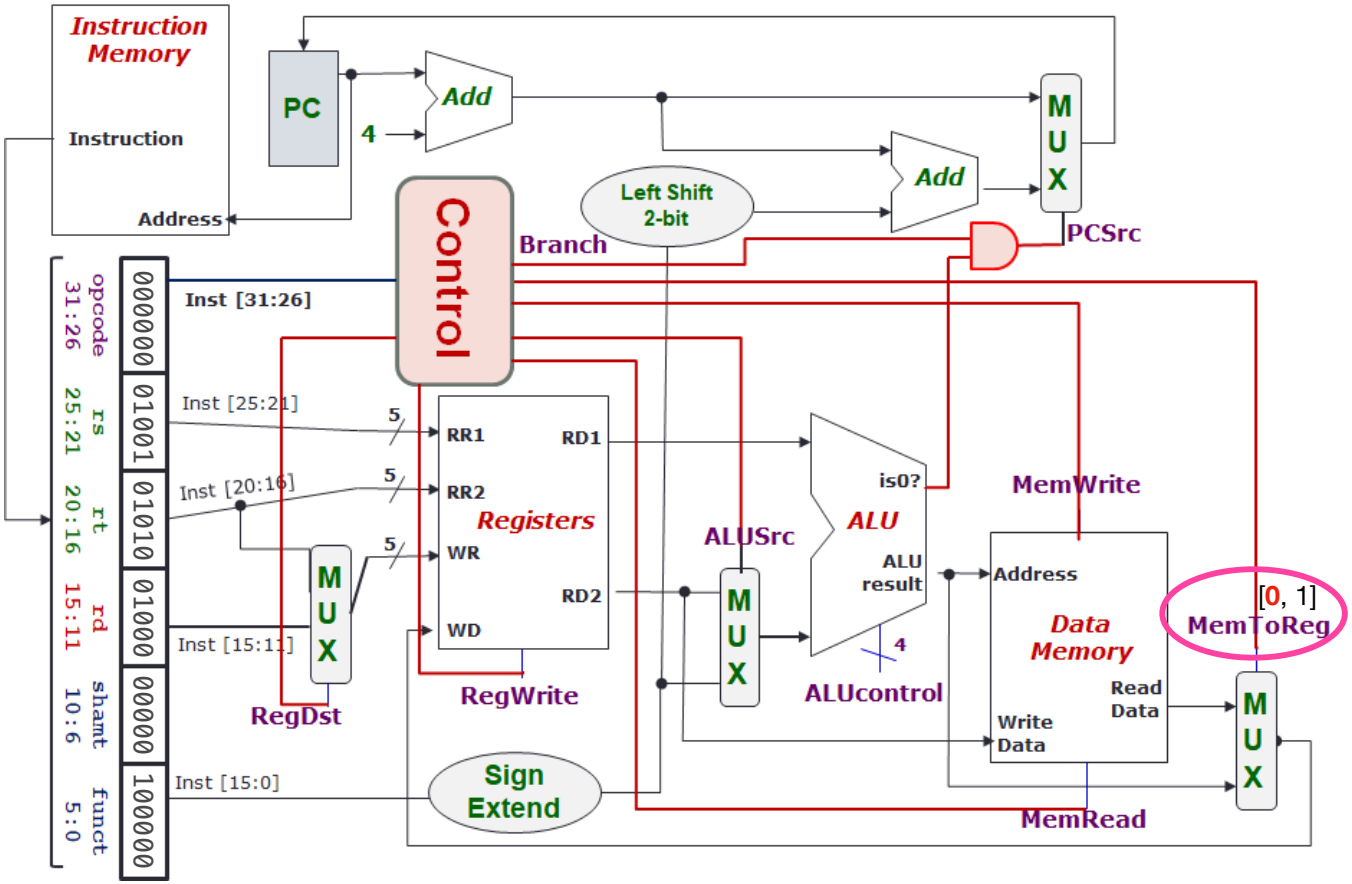


Walkthrough of R-Type Datapath

Unit 9: I-Type vs. R-Type Registers Usages

4: Write-Back:

- The **ALU result** (the sum of `$t1` and `$t2`) is fed into the final multiplexer (MUX labeled **MemToReg**).
- Because the **Control unit** set **MemToReg=0**, this MUX selects the **ALU result**.

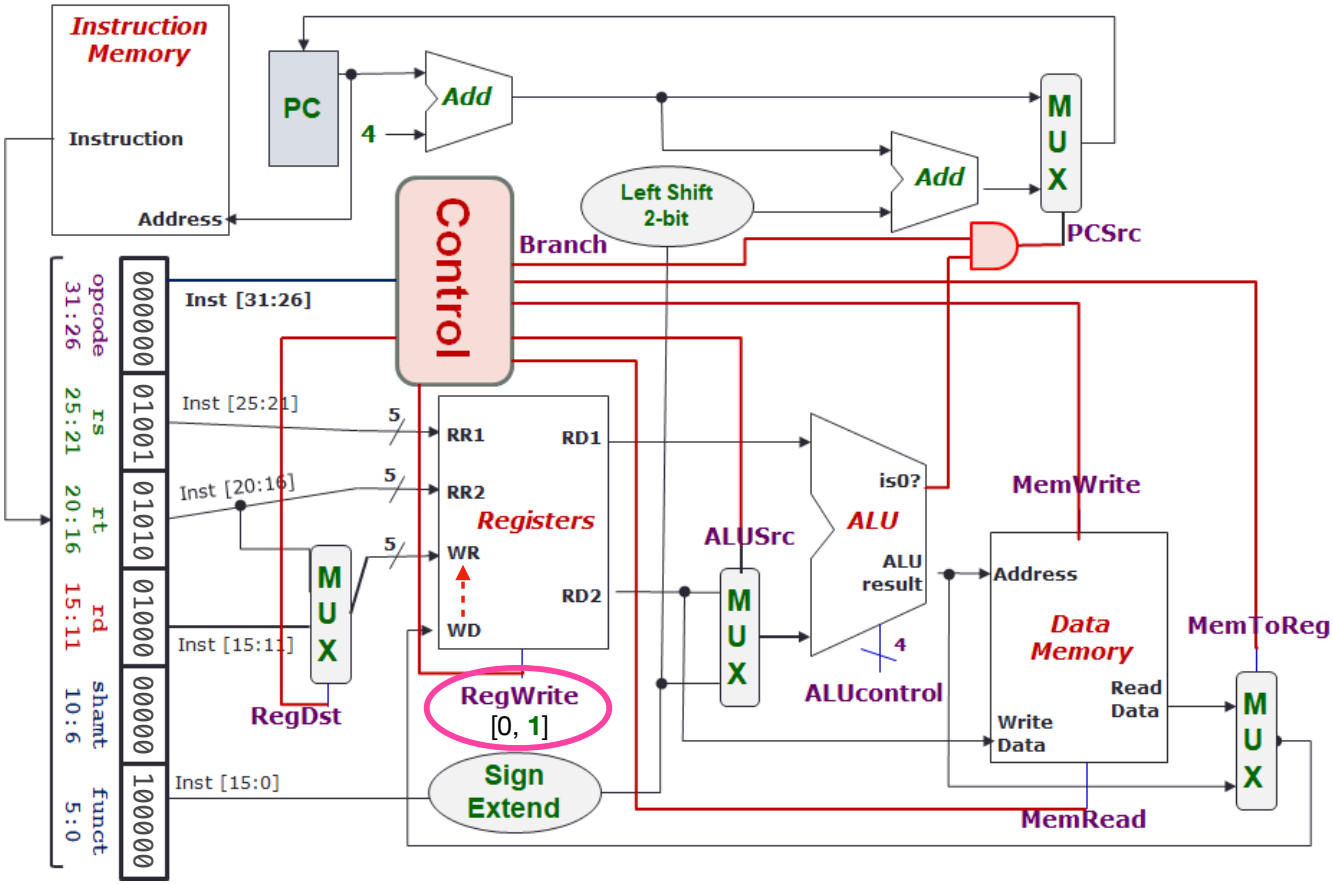


Walkthrough of R-Type Datapath

Unit 9: I-Type vs. R-Type Registers Usages

4: Write-Back:

- This selected value is then fed back as **Write Data** (**WD**) to the **Register File**.
- On the next clock cycle, with **RegWrite**=1, the value on **WD** is written into the register specified on the **Write Register** (**WR**) input, which is `$t0` (8).

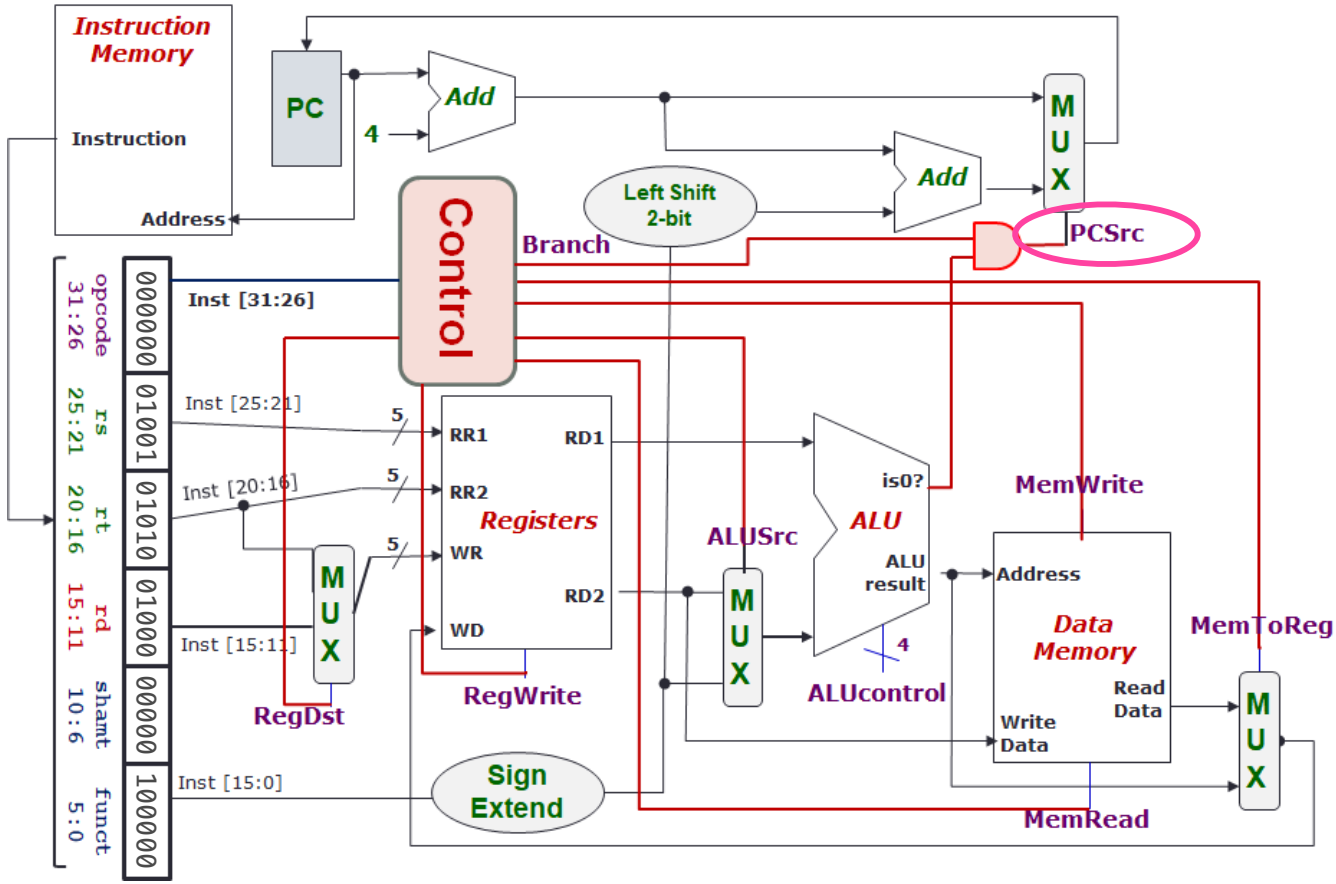


Walkthrough of R-Type Datapath

Unit 9: I-Type vs. R-Type Registers Usages

4: Write-Back:

- PCSrc
  - False (0): $PC' = $PC+4.
  - True (1): $PC' = ($PC+4)+ (immediate×4).

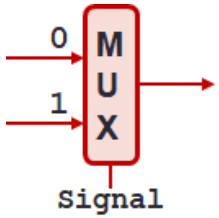


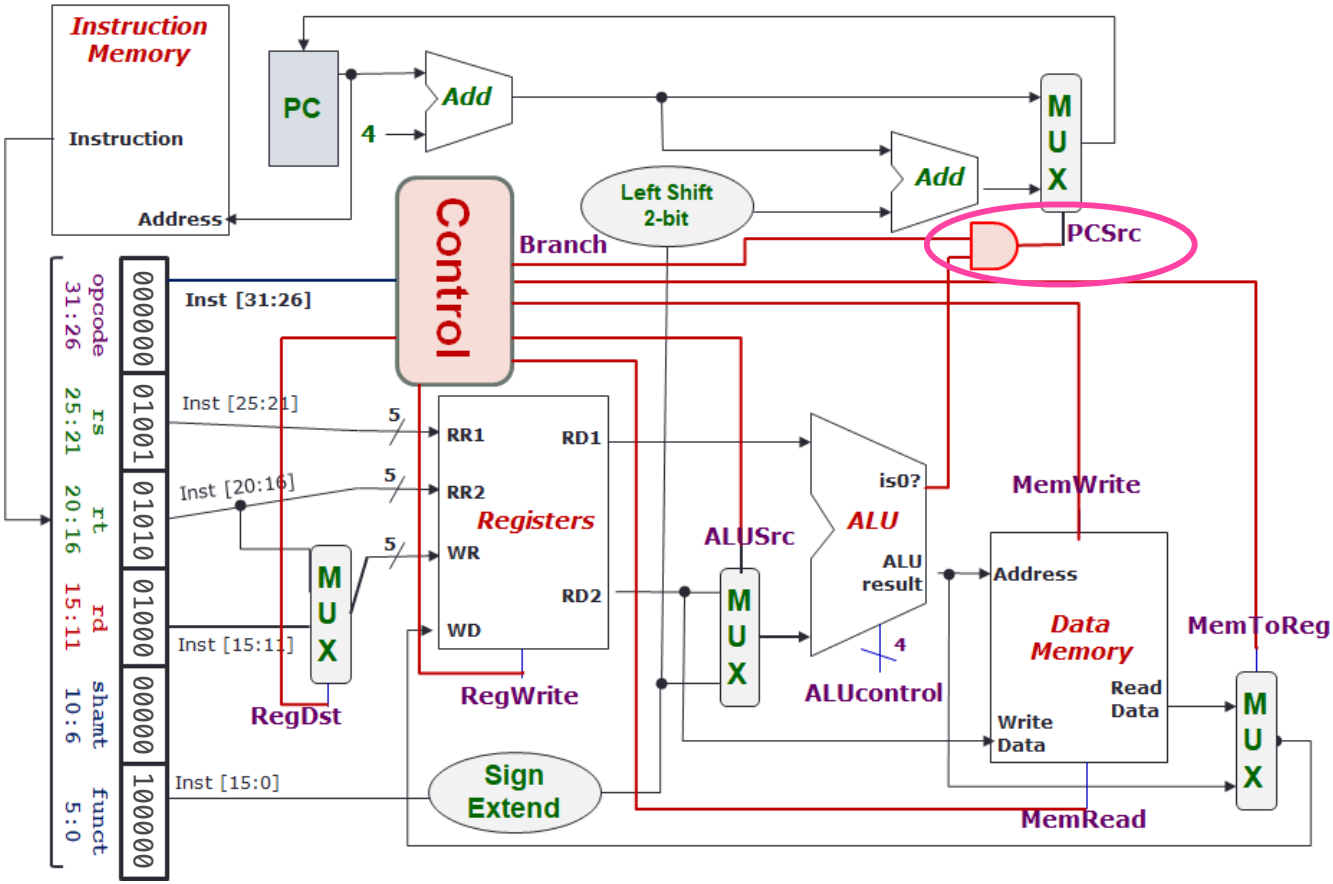
Walkthrough of R-Type Datapath

Unit 9: I-Type vs. R-Type Registers Usages

4: Write-Back:

- Branch
  - False (0): The operation is not a branch operation.
  - True (1): The operation is a branch operation (i.e., beq).
  - PCSrc will be used when applying Branch or/and isZero?





I-Type Instruction Format

I-Type Instructions

Unit 9: I-Type vs. R-Type Registers Usages

- I-Type (Immediate-Type)
  - Purpose: For operations that use a constant value (immediate) embedded in the instruction itself, e.g., `addi $t1, $t2, 25`.
  - Format: 

  - Why **`Op`** is NOT `0`:
    - For I-type instructions, the **`Op`** field uniquely identifies the instruction.
    - There is no **`funct`** field.
    - The CPU sees the **`Op`** code and knows exactly what to do.
- Example: **`Op`** code `001000` means **`addi`**, `001101` means **`ori`**.
    - **`Rs`**: Source register
    - **`Rt`**: Destination register (notice this is different from R-type's **`Rd`**)
    - **`immediate`**: A 16-bit constant value

I-Type Instruction Format

Unit 9: I-Type vs. R-Type Registers Usages

- Constants are used quite frequently in programs.
  - The R-type shift instructions have a 5-bit shift amount constant.
  - What about other instructions that need a constant?
- I-Type: Instructions with **`immediate`** Operands.



- A 16-bit immediate constant is stored inside the instruction
- **`Rs`** is the source register number
- **`Rt`** is now the destination register number (for R-type, it was **`Rd`**)
- Examples of I-Type ALU Instructions:
- Add immediate: `addi $t1, $t2, 5` `# $t1 = $t2 + 5`
- OR immediate: `ori $t1, $t2, 5` `# $t1 = $t2 | 5`

I-Type ALU Instructions

Unit 9: I-Type vs. R-Type Registers Usages

MIPS doesn't have a **subi** instruction!

Instruction	Meaning	Op	Rs	Rt	Immediate
addi $t1, $t2, 25	**$t1 = $t2 + 25**	0x8	$t2	$t1	25
addiu $t1, $t2, 25	**$t1 = $t2 + 25**	0x9	$t2	$t1	25
andi $t1, $t2, 25	**$t1 = $t2 & 25**	0xc	$t2	$t1	25
ori $t1, $t2, 25	**$t1 = $t2 | 25**	0xd	$t2	$t1	25
xori $t1, $t2, 25	**$t1 = $t2 ^ 25**	0xe	$t2	$t1	25
lui $t1, 25	**$t1 = 25 << 16**	0xf	0	$t1	25

- **addi**: overflow causes an arithmetic exception
  - In case of overflow, the result is not written to the destination register
- **addiu**: same operation as **addi**, but overflow is ignored
- Immediate constant for **addi** and **addiu** is signed
  - No need for **subi** or **subiu** instructions
- Immediate constant for **andi**, **ori**, **xori** is unsigned

I-Type ALU Instructions

Unit 9: I-Type vs. R-Type Registers Usages

Instruction	Meaning	Op	Rs	Rt	Immediate
`addi $t1, $t2, 25`	`$t1 = $t2 + 25`	0x8	$t2	$t1	25
`addiu $t1, $t2, 25`	`$t1 = $t2 + 25`	0x9	$t2	$t1	25
`andi $t1, $t2, 25`	`$t1 = $t2 & 25`	0xc	$t2	$t1	25
`ori $t1, $t2, 25`	`$t1 = $t2 | 25`	0xd	$t2	$t1	25
`xori $t1, $t2, 25`	`$t1 = $t2 ^ 25`	0xe	$t2	$t1	25
`lui $t1, 25`	`$t1 = 25 << 16`	0xf	0	$t1	25

- `lui $t1, 0x1234` loads the 16-bit `immediate` value into the upper half (the most significant 16 bits) of the destination register `$t1` and sets the lower 16 bits to zero.
- Result: `$t1` becomes `0x12340000`.
- Since the `immediate` field in I-type instructions is only 16 bits, we can't load a full 32-bit constant (like a memory address) in one instruction.

I-Type ALU Instructions

Unit 9: I-Type vs. R-Type Registers Usages

Instruction	Meaning	Op	Rs	Rt	Immediate
`addi $t1, $t2, 25`	`$t1 = $t2 + 25`	0x8	$t2	$t1	25
`addiu $t1, $t2, 25`	`$t1 = $t2 + 25`	0x9	$t2	$t1	25
`andi $t1, $t2, 25`	`$t1 = $t2 & 25`	0xc	$t2	$t1	25
`ori $t1, $t2, 25`	`$t1 = $t2 | 25`	0xd	$t2	$t1	25
`xori $t1, $t2, 25`	`$t1 = $t2 ^ 25`	0xe	$t2	$t1	25
`lui $t1, 25`	`$t1 = 25 << 16`	0xf	0	$t1	25

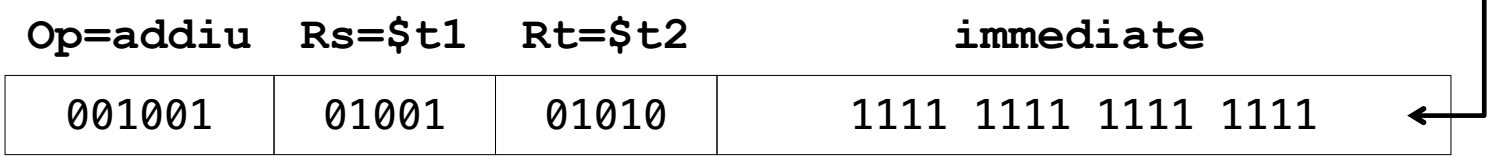
- `lui` is the first step of a two-step process:
  - `lui $t1, 0x1001` # Load the upper part of the address: `$t1 = 0x10010000`
  - `ori $t1, $t1, 0x1234`
    # OR in the lower part: `$t1 = 0x10010000 | 0x00001234 = 0x10011234`

- Now `$t1` holds the full 32-bit address `0x10011234`.

Examples of I-Type ALU Instructions

Unit 9: I-Type vs. R-Type Registers Usages

Expression	Equivalent MIPS Instruction
`A = B + 5;`	`addiu $t0, $t1, 5`
`C = B - 1;`	`addiu $t2, $t1, -1`
`A = B & 0xf;`	`andi  $t0, $t1, 0xf`
`C = B | 0xf;`	`ori   $t2, $t1, 0xf`
`C = 5;`	`addiu $t2, $zero, 5`
`A = B;`	`addiu $t0, $t1, 0`



- No need for **`subiu`**, because **`addiu`** has signed **`immediate`**
- Register `$zero` has always the value `0`

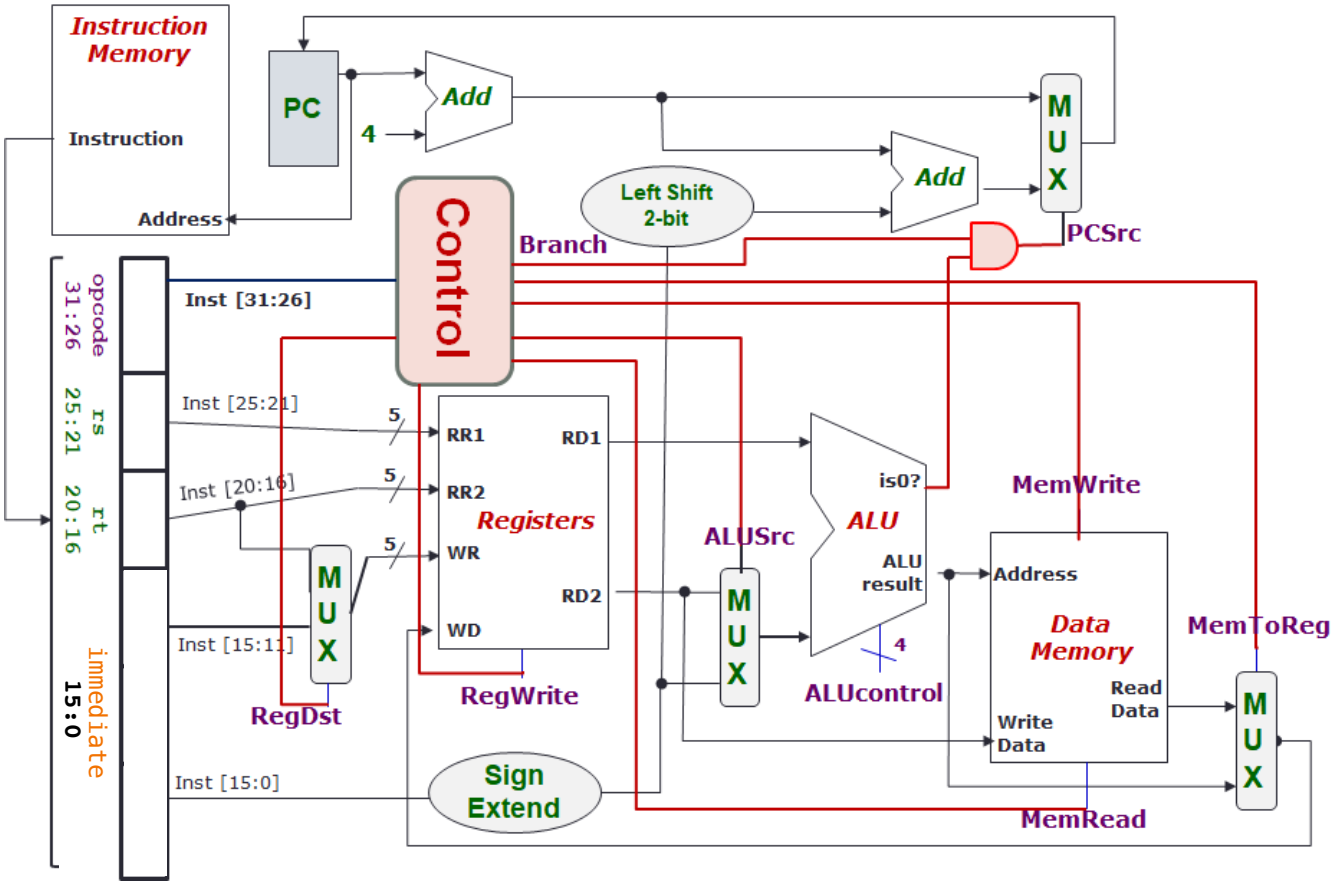
Walkthrough of I-Type Datapath

Unit 9: I-Type vs. R-Type Registers Usages

- Let's follow the path of an `addiu $t2, $t1, -1` instruction through this diagram.
- `$t1` is register 9 and `$t2` is register 10.

opcode (6)	rs (5)	rt (5)	immediate (16)

Instruction		Opcode	Notes
addi	rt, rs, immediate	001000	
addiu	rt, rs, immediate	001001	
andi	rt, rs, immediate	001100	



Walkthrough of I-Type Datapath

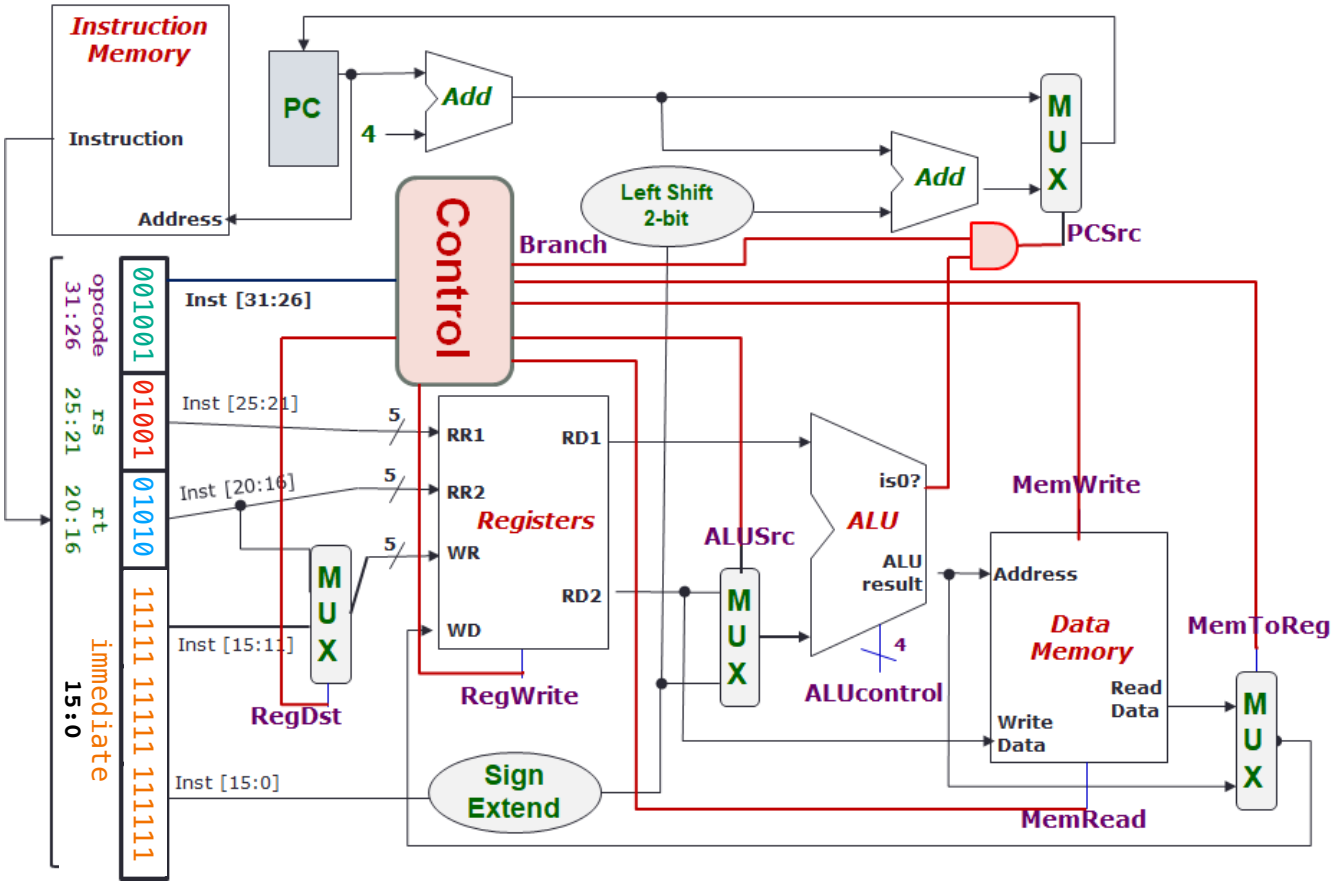
Unit 9: I-Type vs. R-Type Registers Usages

opcode (6)	rs (5)	rt (5)	immediate (16)

Instruction		Opcode	Notes
addi	rt, rs, immediate	001000	
addiu	rt, rs, immediate	001001	
andi	rt, rs, immediate	001100	

1: Instruction Fetch:

- The Program Counter (PC) holds the address of the current instruction.
- This address is stored in the **Instruction Memory**.
- The instruction memory outputs the 32-bit instruction **word** `00100101001010101111111111111111` (which is the machine code for `addiu $10, $9, -1`).

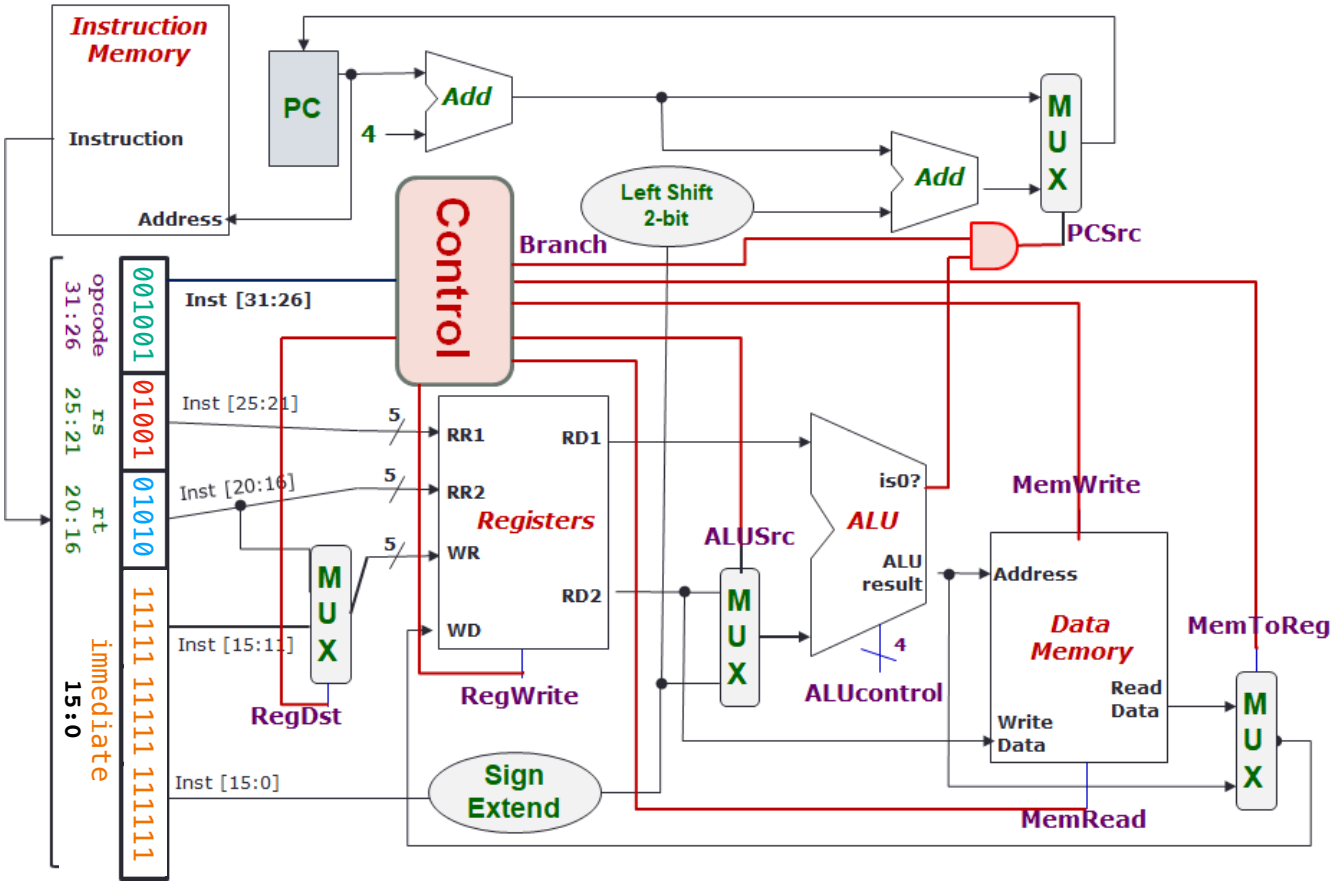


Walkthrough of I-Type Datapath

Unit 9: I-Type vs. R-Type Registers Usages

2: Instruction Decode & Register Fetch:

- The instruction bits are split and sent to different units.
- opcode [31:26] (001001) goes to the **Control unit**.
- rs [25:21] (01001 for $t1or $9) is connected to **Read register 1** (**RR1**) of the register file.
- rt [20:16] (01010 for $t2 or $10) is connected to the **Write register** (**WR**) of the register file.

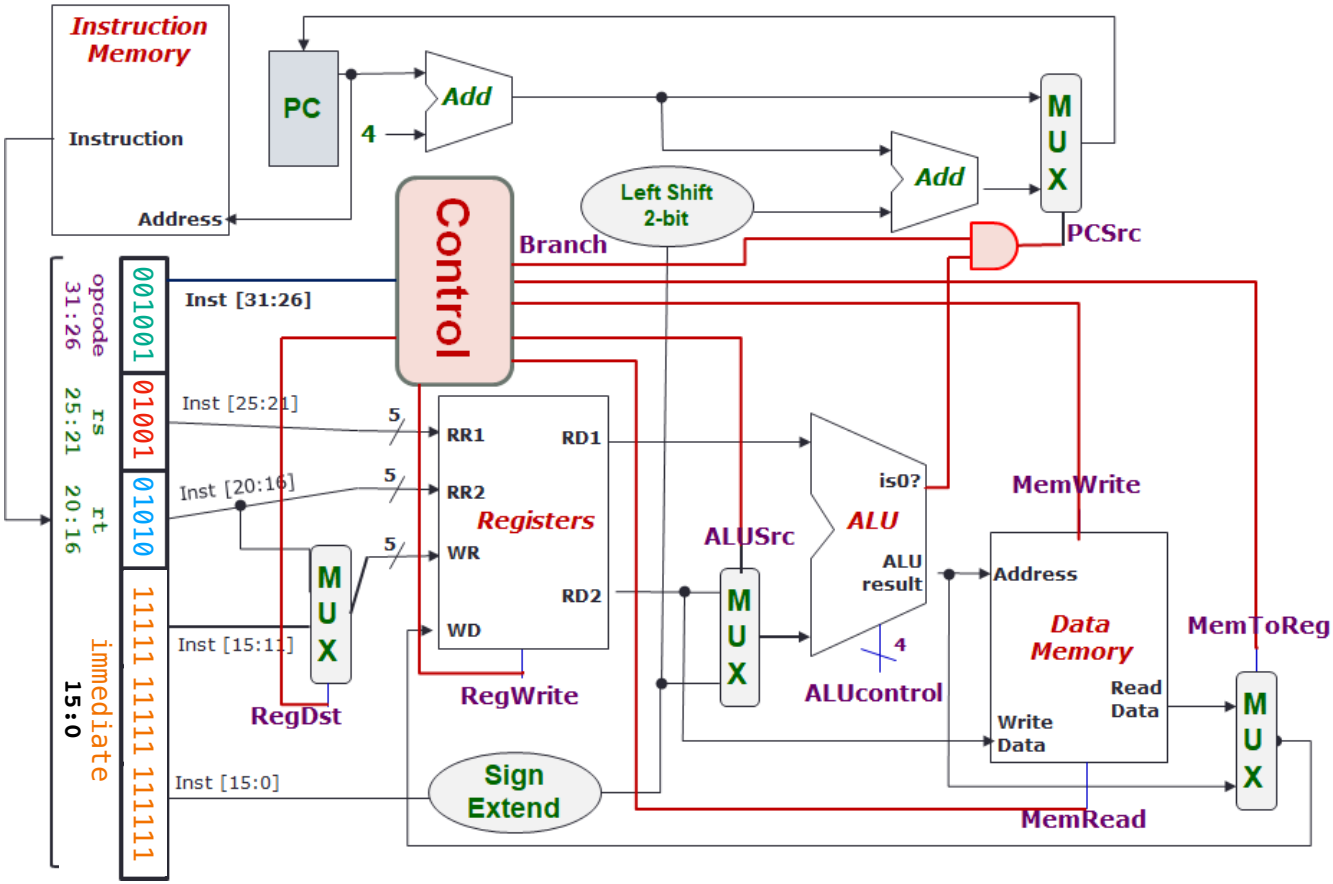


Walkthrough of I-Type Datapath

Unit 9: I-Type vs. R-Type Registers Usages

2: Instruction Decode & Register Fetch:

- `immediate` [15:0] (11111 11111 111111) is connected to the **Sign Extend** input.
- This is a key difference from **R-type** instructions.

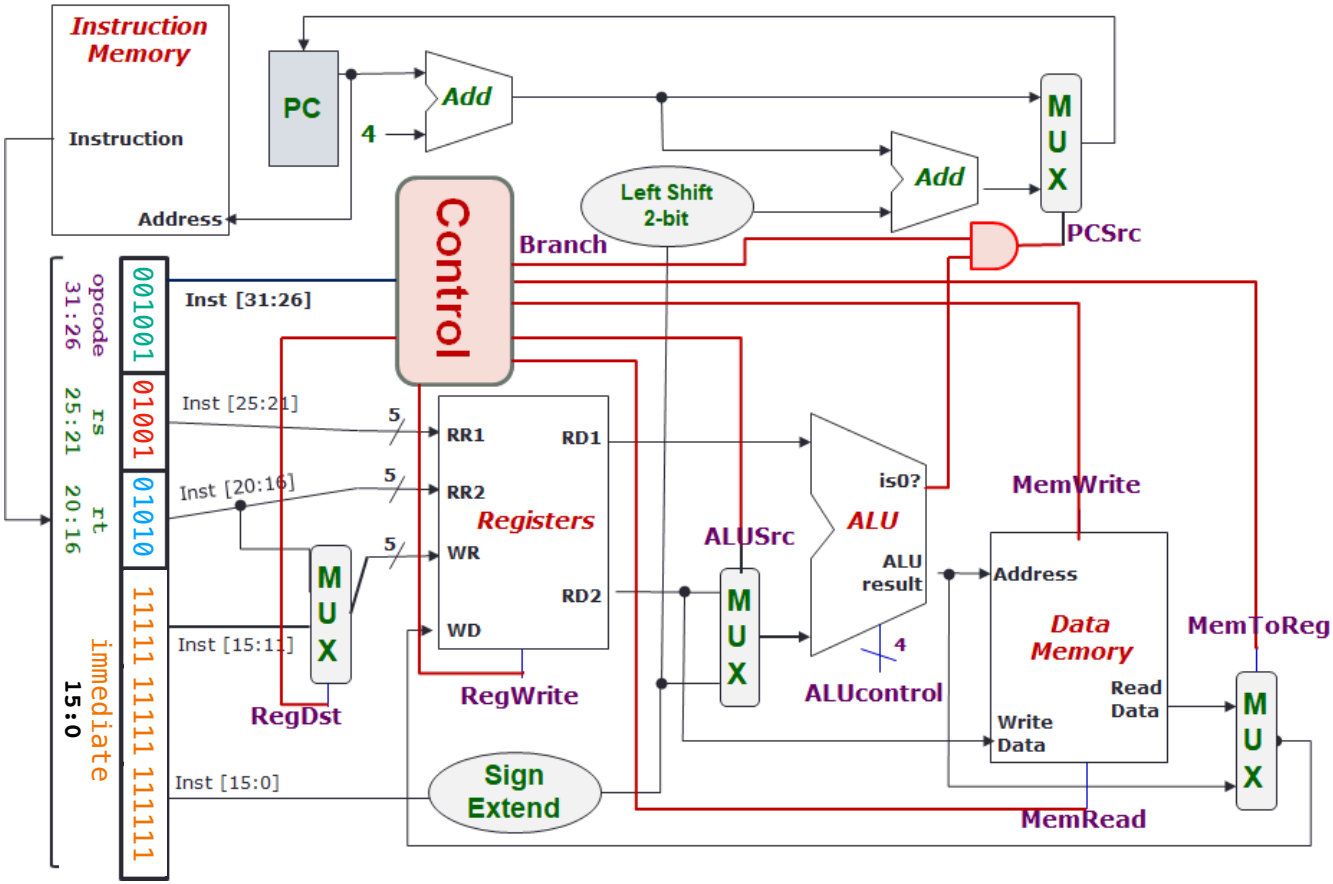


Walkthrough of I-Type Datapath

Unit 9: I-Type vs. R-Type Registers Usages

2: Instruction Decode & Register Fetch:

- The **Control unit** sees the opcode is `001001` (**I-type**) and sets its output signals accordingly.

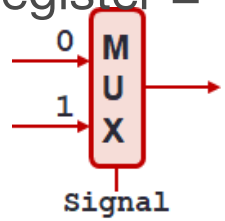


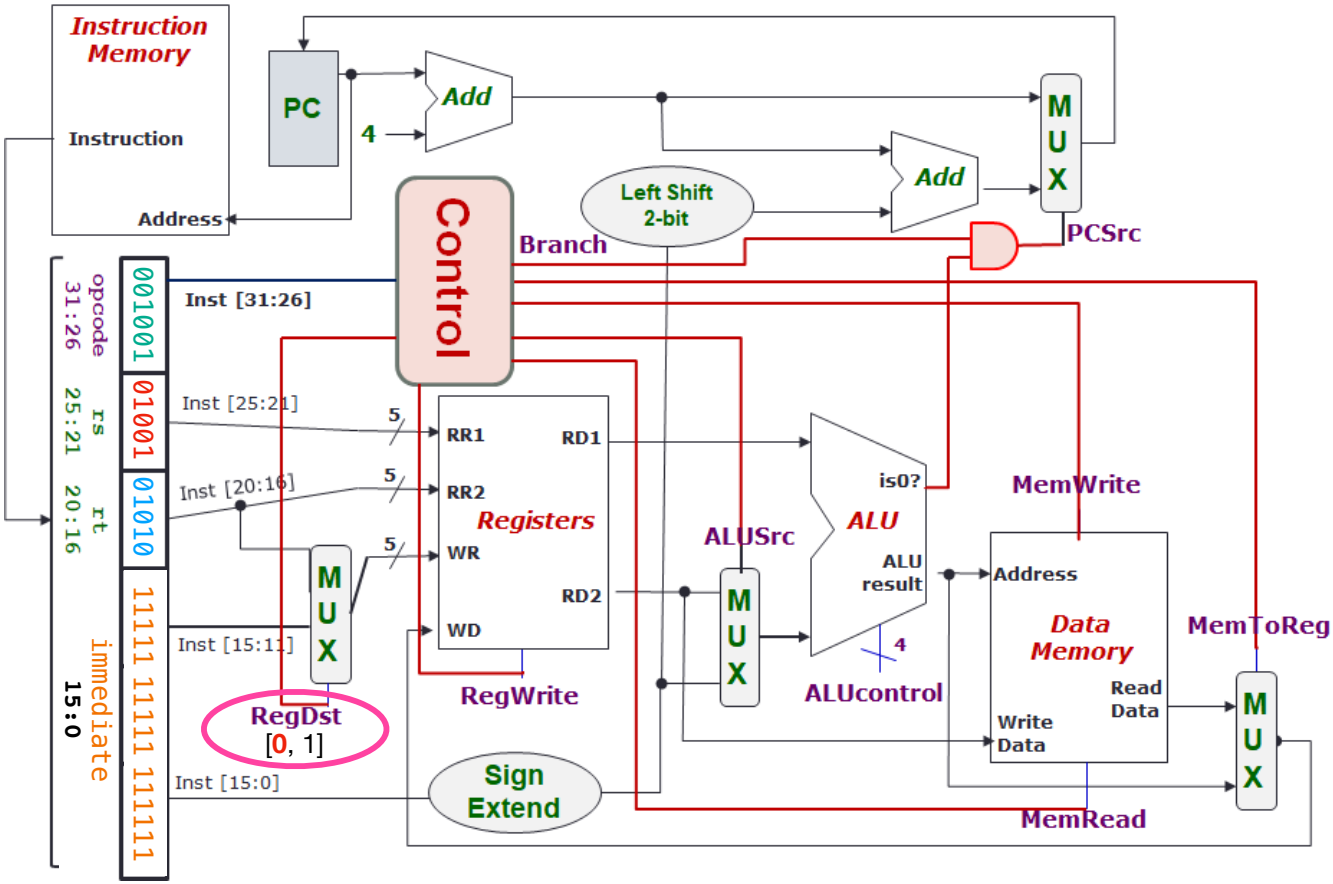
Walkthrough of I-Type Datapath

Unit 9: I-Type vs. R-Type Registers Usages

2: Instruction Decode & Register Fetch:

- **RegDst** = 0; this controls a MUX, instructing it to use the `rt` field from the instruction [20:16] as the write register (rather than `rd` [15:11] in R-Type).
  - False (0): Write Register = Inst[20:16] **`rt`**
  - True (1): Write Register = Inst[15:11] **`rd`**



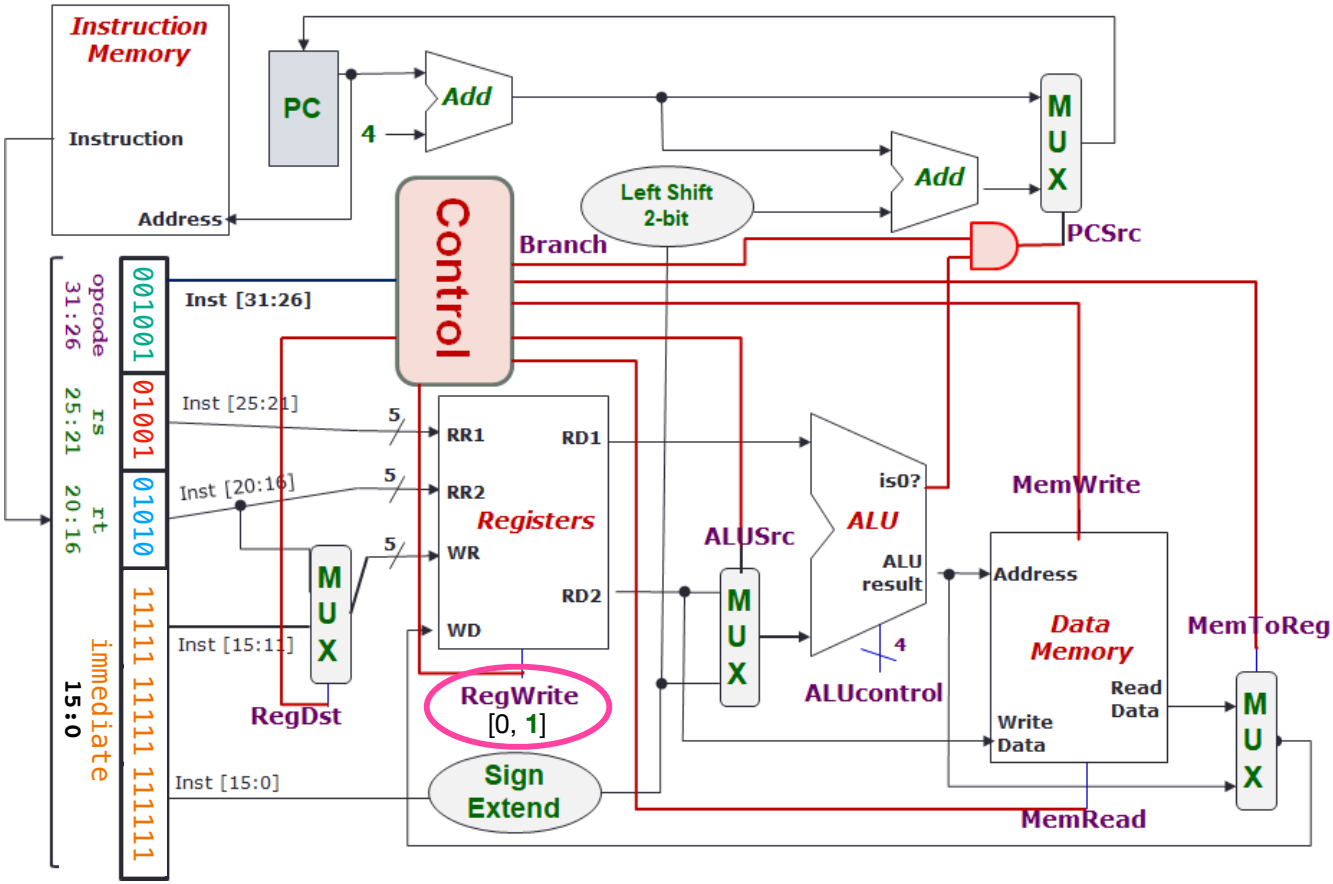


Walkthrough of I-Type Datapath

Unit 9: I-Type vs. R-Type Registers Usages

2: Instruction Decode & Register Fetch:

- **RegWrite** = 1, because we will write to a register at the end.
  - False (0): No register write
  - True (1): New value will be written

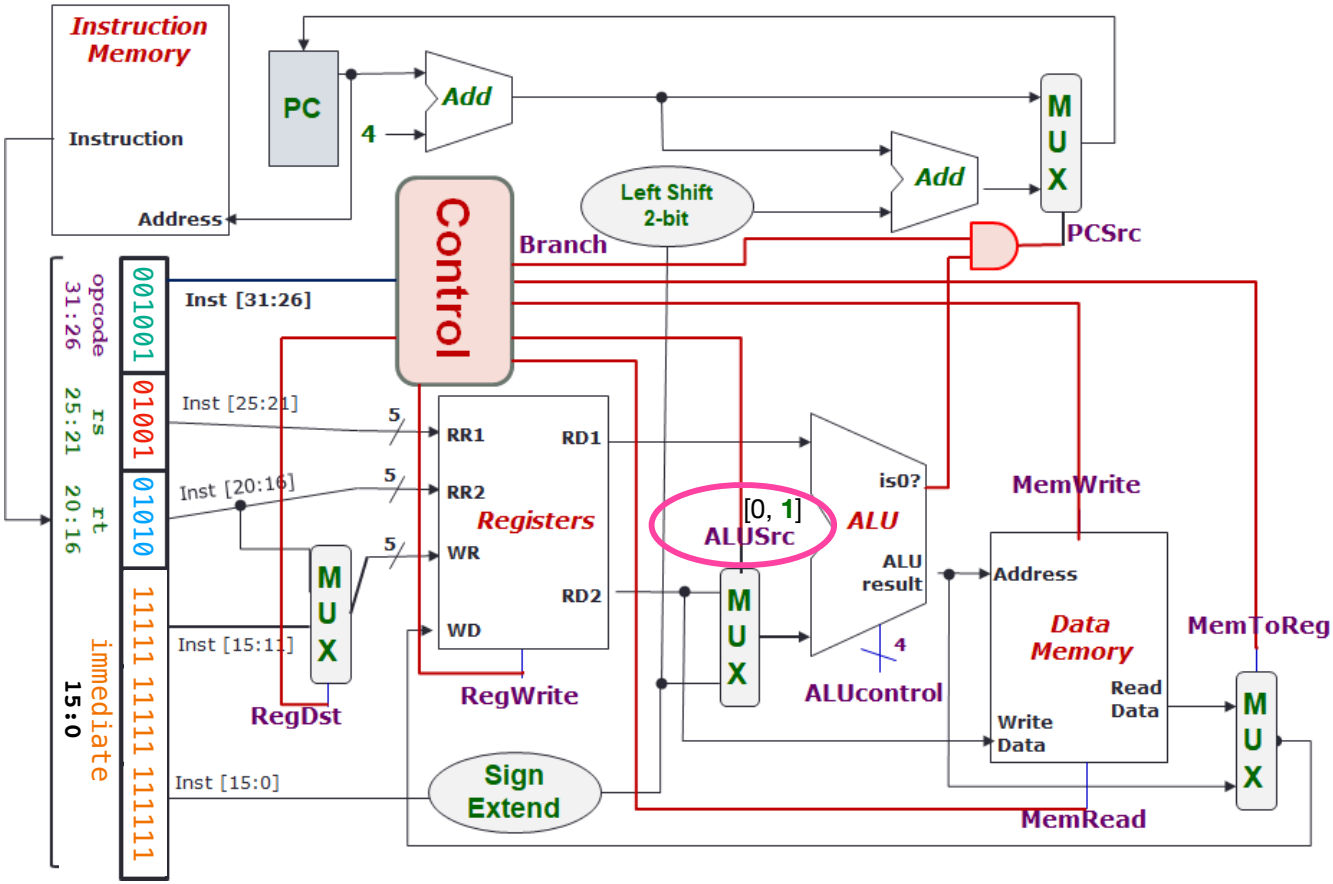


Walkthrough of I-Type Datapath

Unit 9: I-Type vs. R-Type Registers Usages

2: Instruction Decode & Register Fetch:

- **ALUSrc** = 1; this controls a MUX, instructing it to use the sign-extended immediate value as the second ALU input, rather than the second register read from the register file.
  - False (0): ALUop2 = Read Data 2
  - True (1): ALUop2 = sign_extend(Inst[15:0])

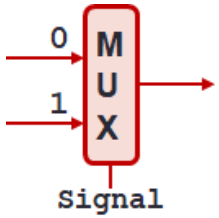


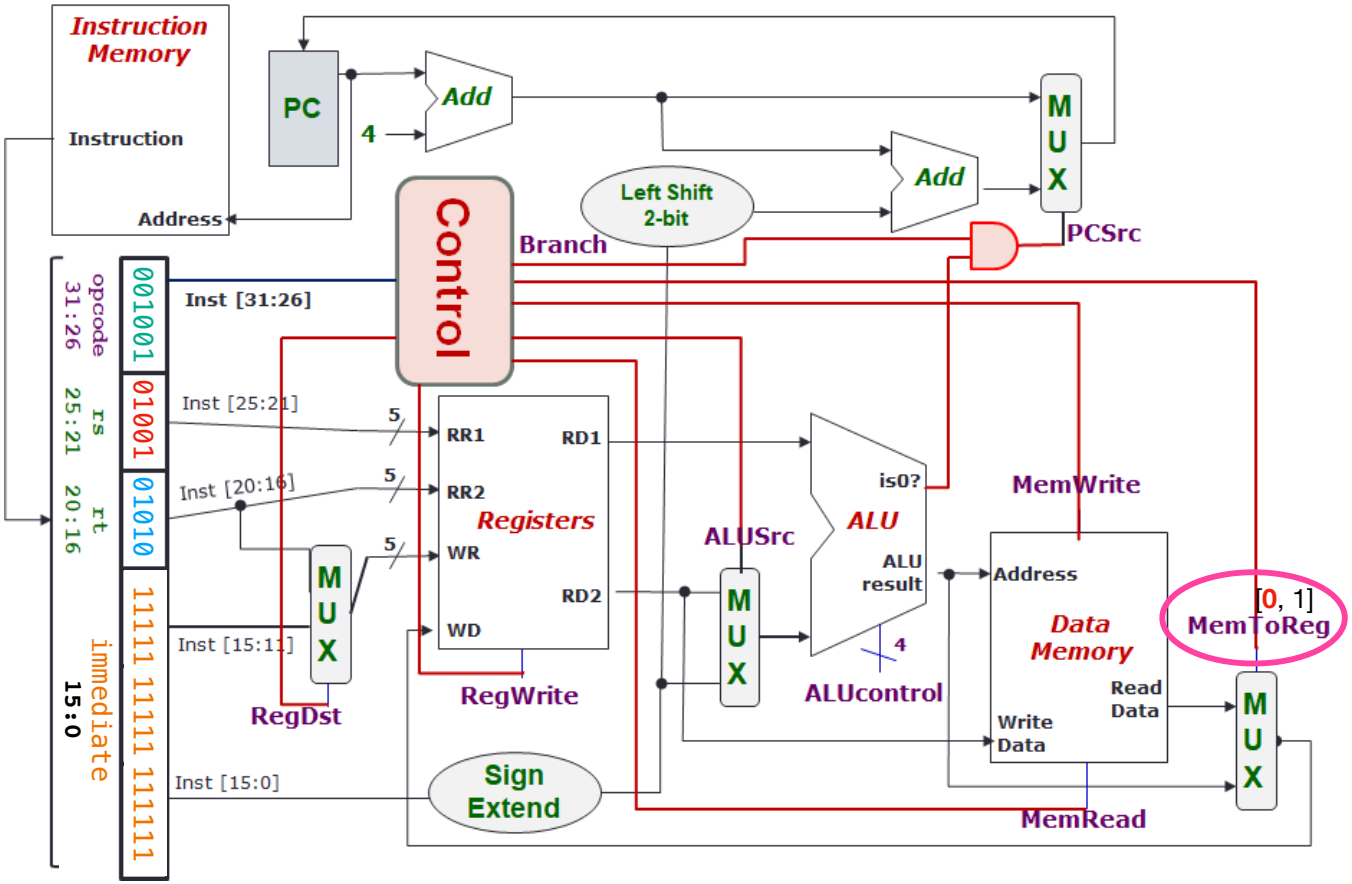
Walkthrough of I-Type Datapath

Unit 9: I-Type vs. R-Type Registers Usages

2: Instruction Decode & Register Fetch:

- **MemToReg** = 0, this controls the final MUX, telling it to write the ALU result back to the register file, not the data from memory.
  - True (1): Register Write Data = Memory Read Data.
  - False (0): Register Write Data = **ALU result**.



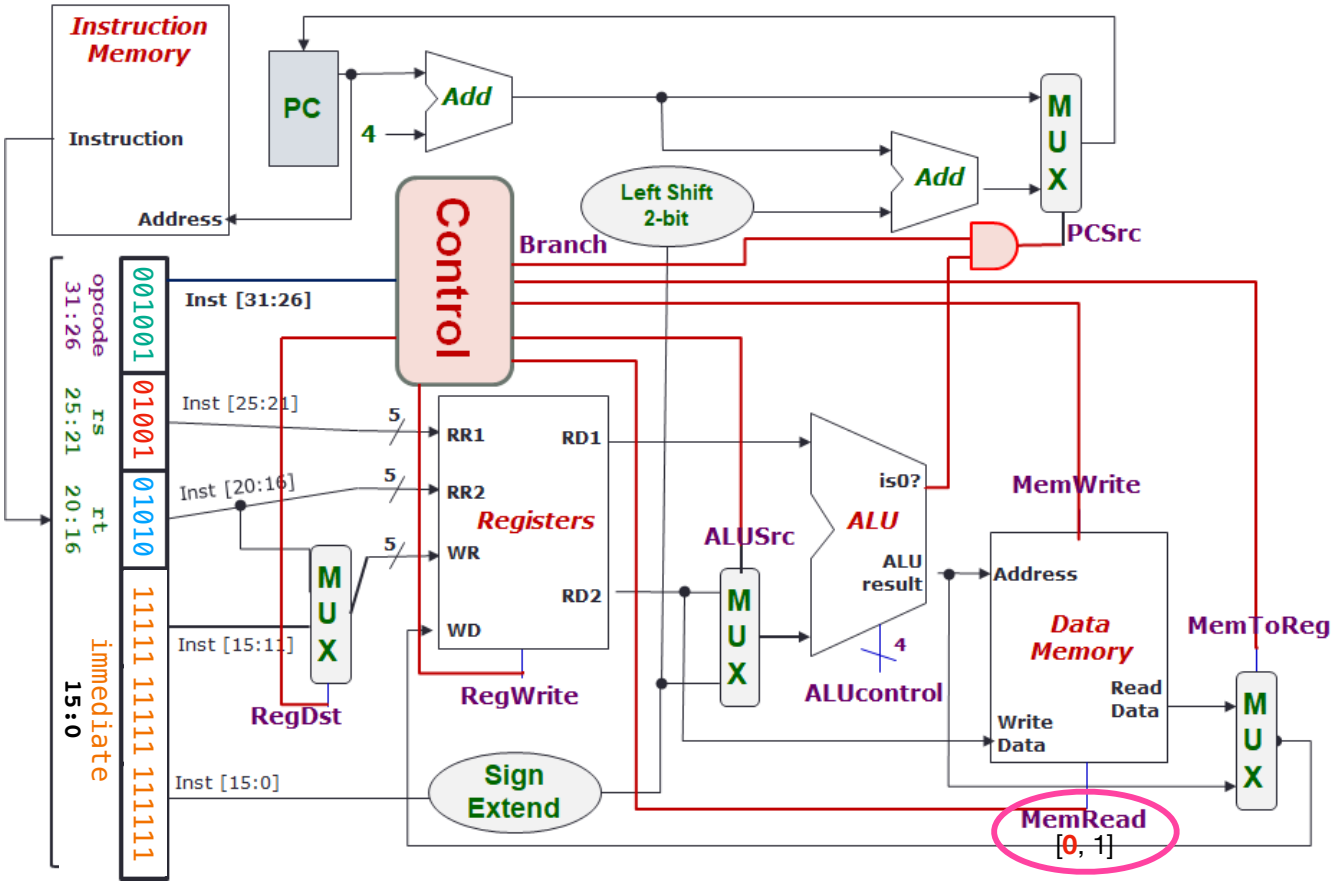


Walkthrough of I-Type Datapath

Unit 9: I-Type vs. R-Type Registers Usages

2: Instruction Decode & Register Fetch:

- **MemRead** = 0
  - False (0): Not performing memory read access.
  - True (1): Read memory using ALU result as **Address**.

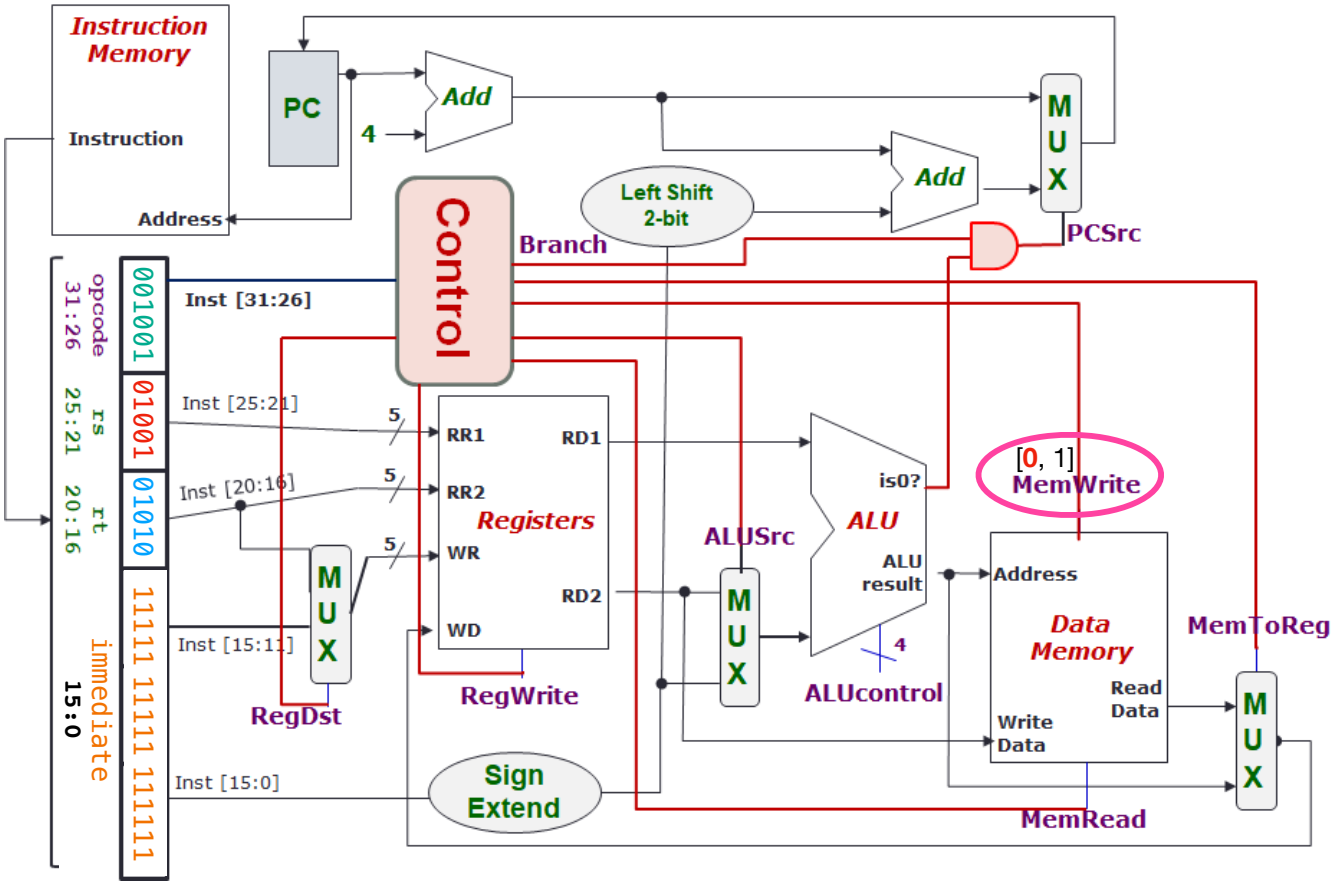


Walkthrough of I-Type Datapath

Unit 9: I-Type vs. R-Type Registers Usages

2: Instruction Decode & Register Fetch:

- **MemWrite** = 0
  - False (0): Not performing memory write operation.
  - True (1): Write to memory at Address (from ALU result) with data from **Read Data 2**.
- Branch = 0

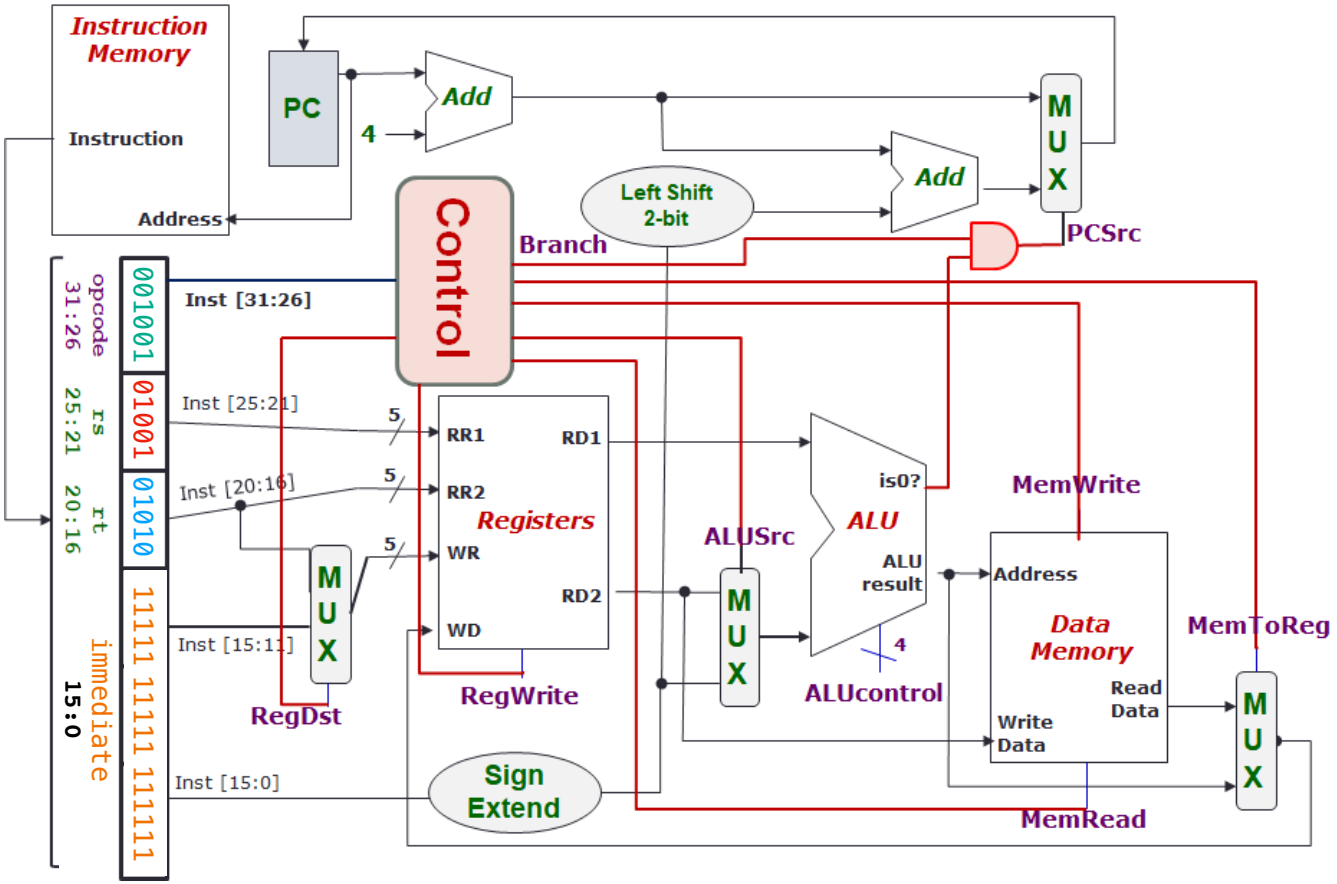


Walkthrough of I-Type Datapath

Unit 9: I-Type vs. R-Type Registers Usages

2: Instruction Decode & Register Fetch:

- The **Register File** reads the value from the register specified by the `rs` field (`$t1`) and outputs it on **Read Data 1** (**RD1**).
- It also reads the value from the register specified by the `rt` field (`$t2`) and outputs it on **Read Data 2** (**RD2**), although this value from `$t2` will not be used by the addiu instruction.

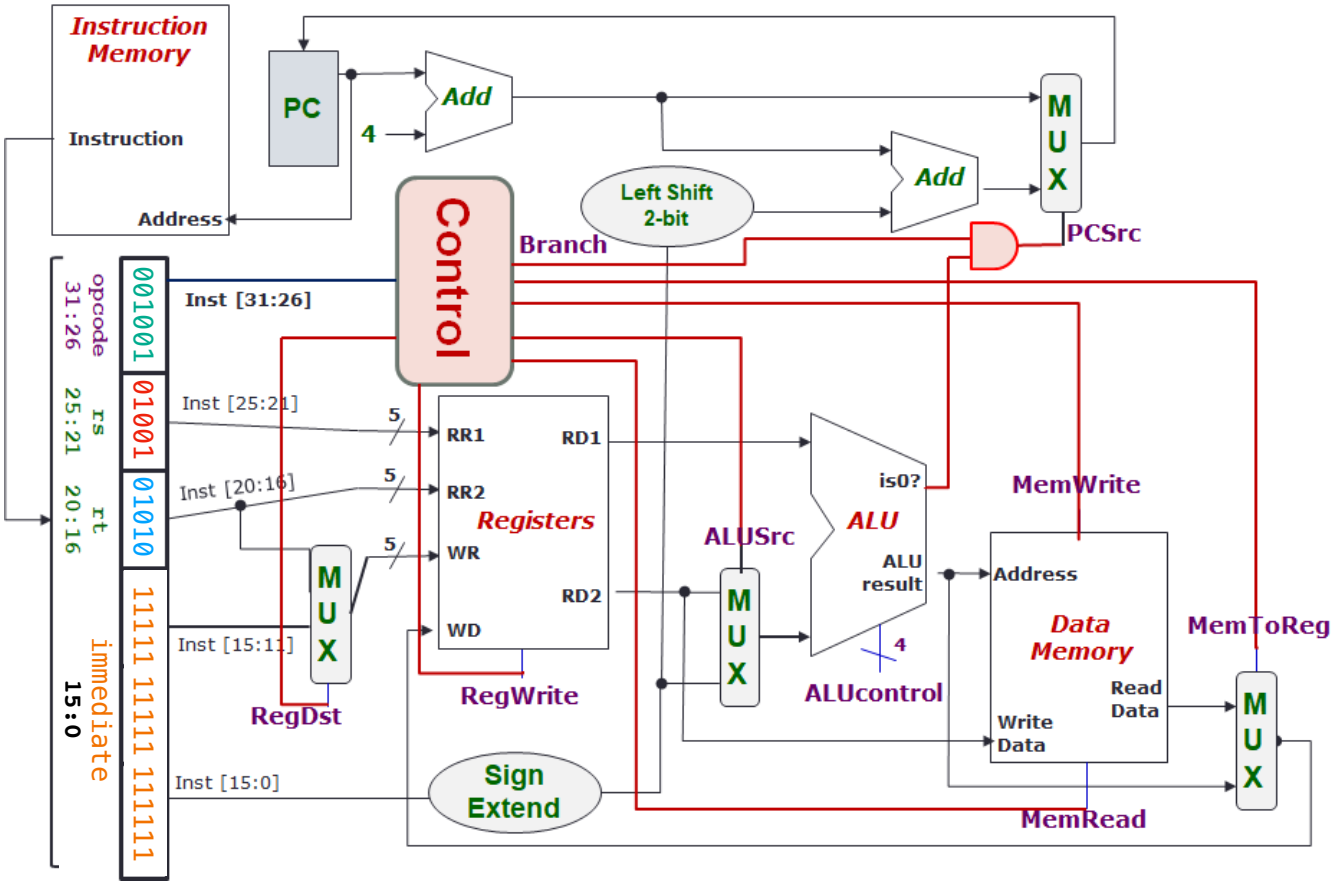


Walkthrough of I-Type Datapath

Unit 9: I-Type vs. R-Type Registers Usages

3: Execution:

- **RD1** (value of `$t1`) and the Sign-Extended Immediate value (-1) are sent to the **ALU**.
- The `funct` field is not used.
- For I-Type instructions, the operation is determined solely by the opcode [31:26].

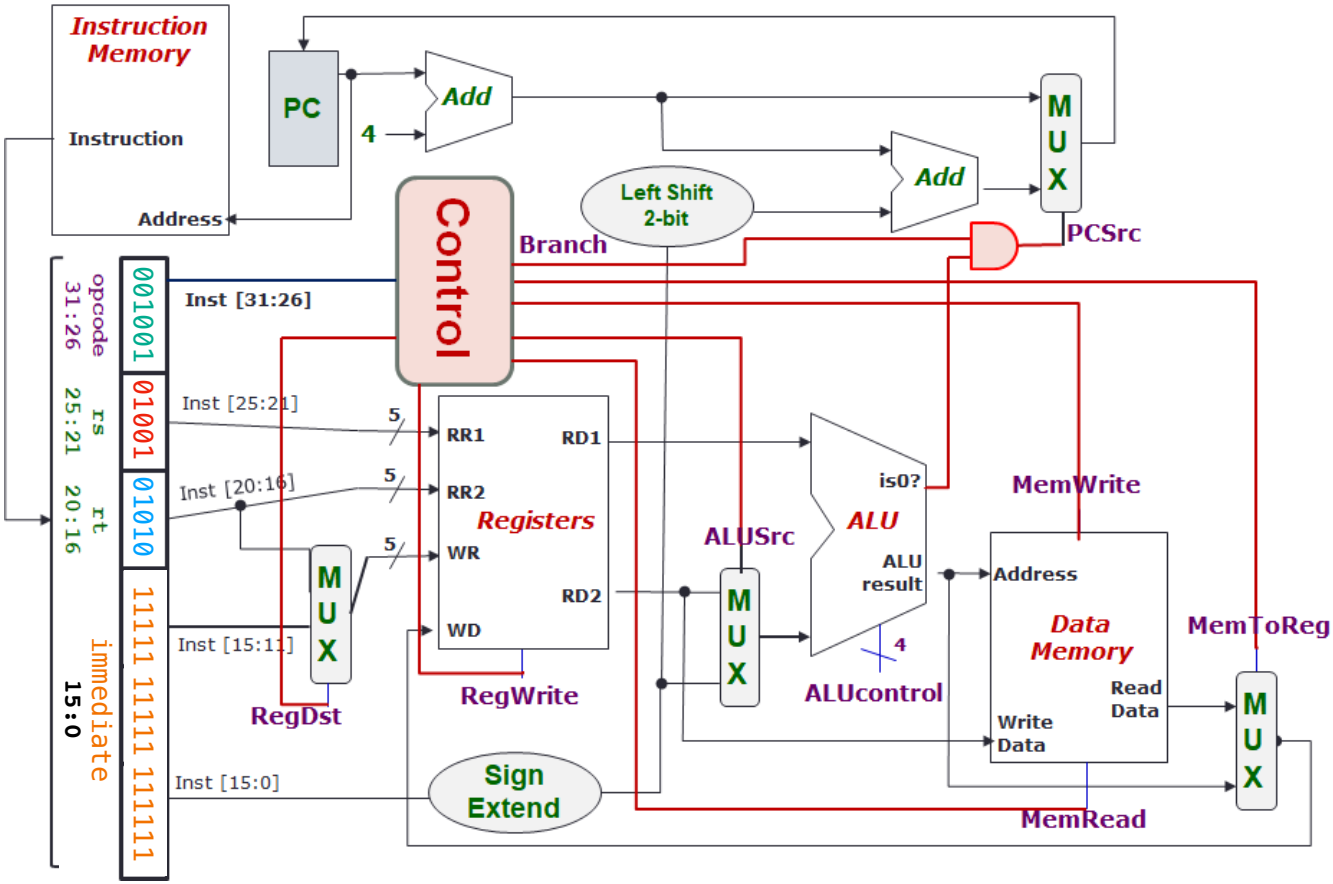


Walkthrough of I-Type Datapath

Unit 9: I-Type vs. R-Type Registers Usages

3: Execution:

- The **Control Unit** decodes the `addiu` opcode (`001001`) and sends the appropriate `ALUOp` signal to the **ALU Control** unit, which in turn commands the main ALU to perform an unsigned addition operation.

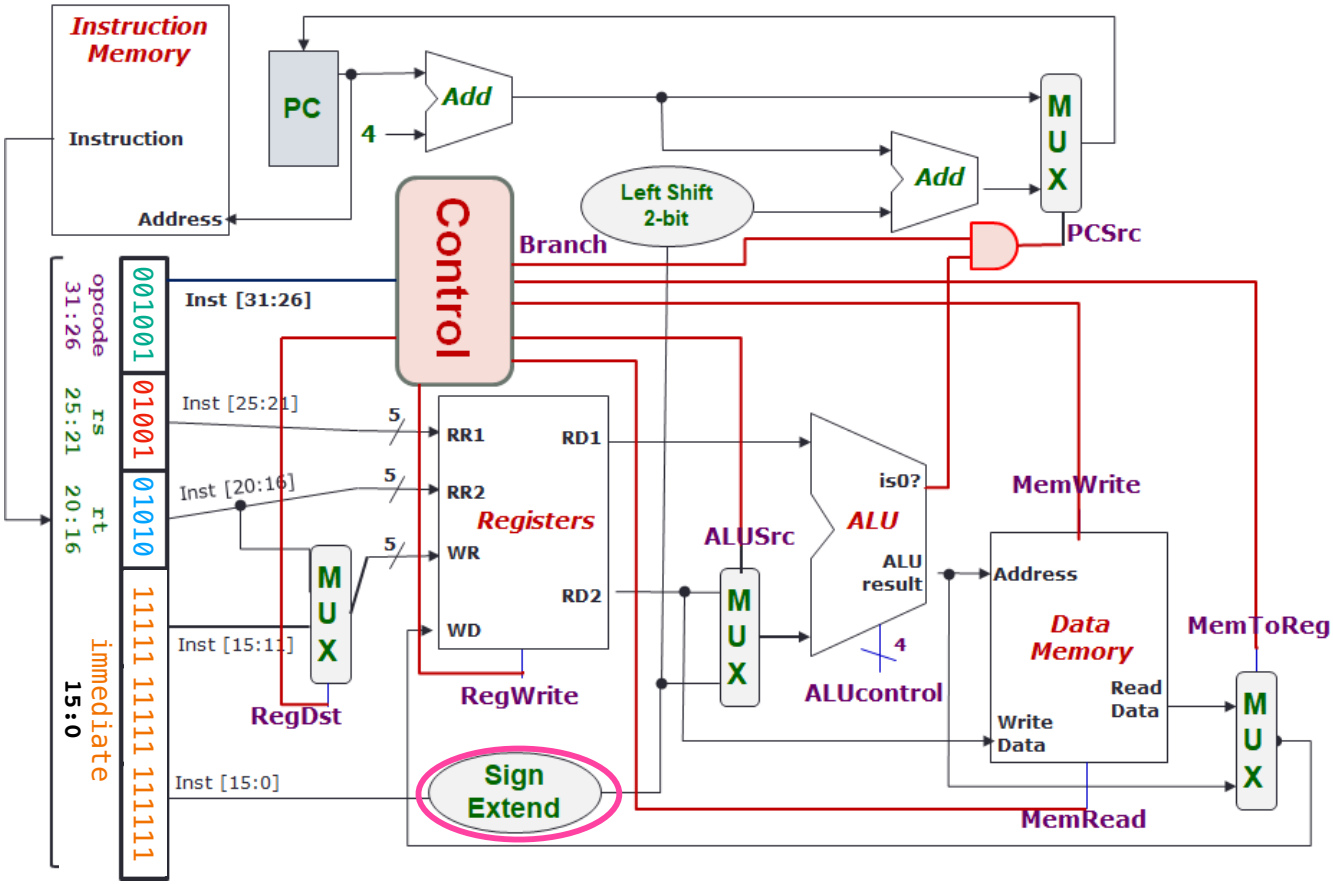


Walkthrough of I-Type Datapath

Unit 9: I-Type vs. R-Type Registers Usages

3: Execution:

- The **ALUSrc** control signal is set to 1.
- This critical signal controls the multiplexer before the ALU's second input, selecting the **Sign-Extended** `Immediate` value instead of the **RD2** value from the register file.

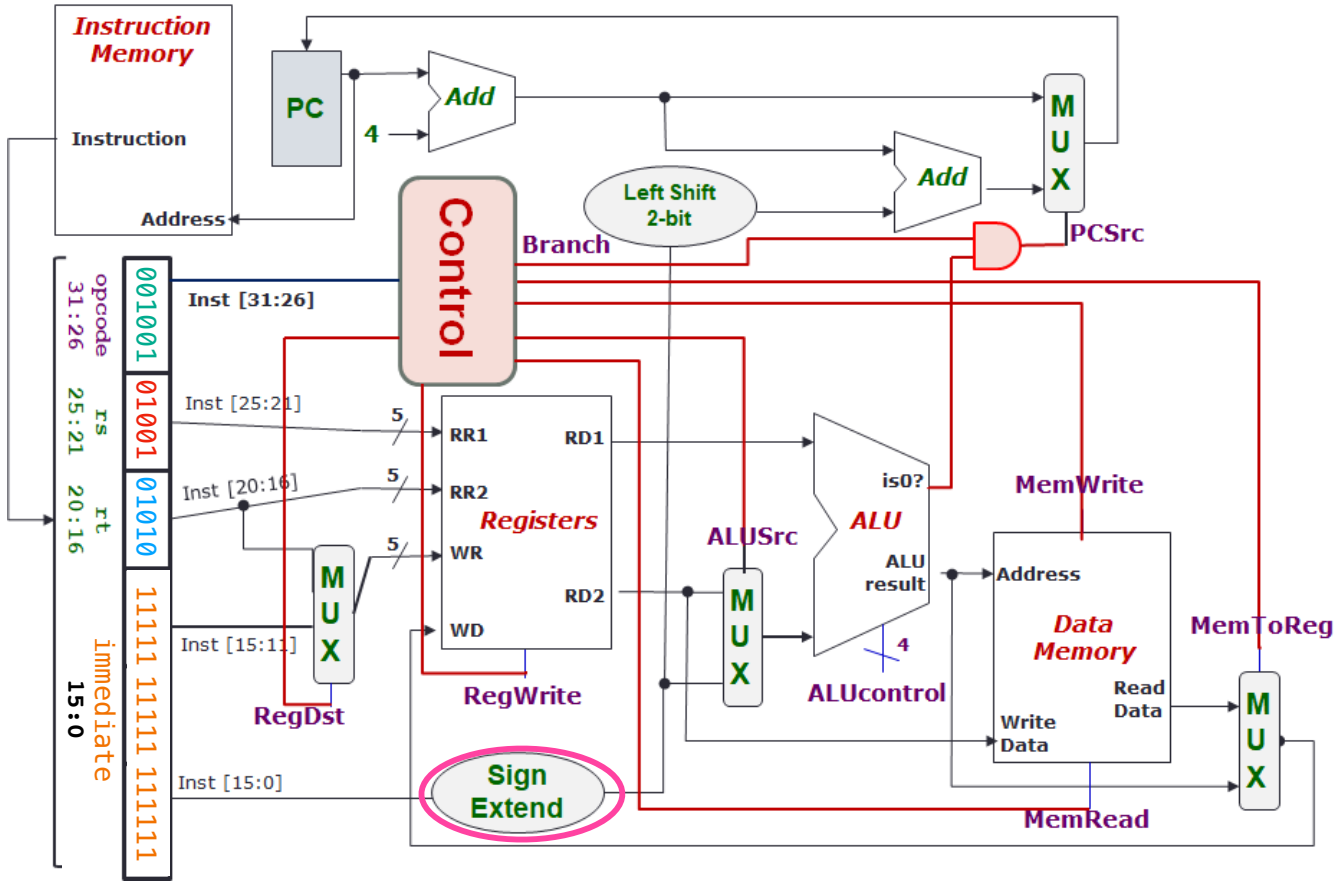


Walkthrough of I-Type Datapath

Unit 9: I-Type vs. R-Type Registers Usages

3: Execution:

- The **ALU** performs the unsigned addition on the value from `$t1` and the `immediate` value −1, and outputs the result.

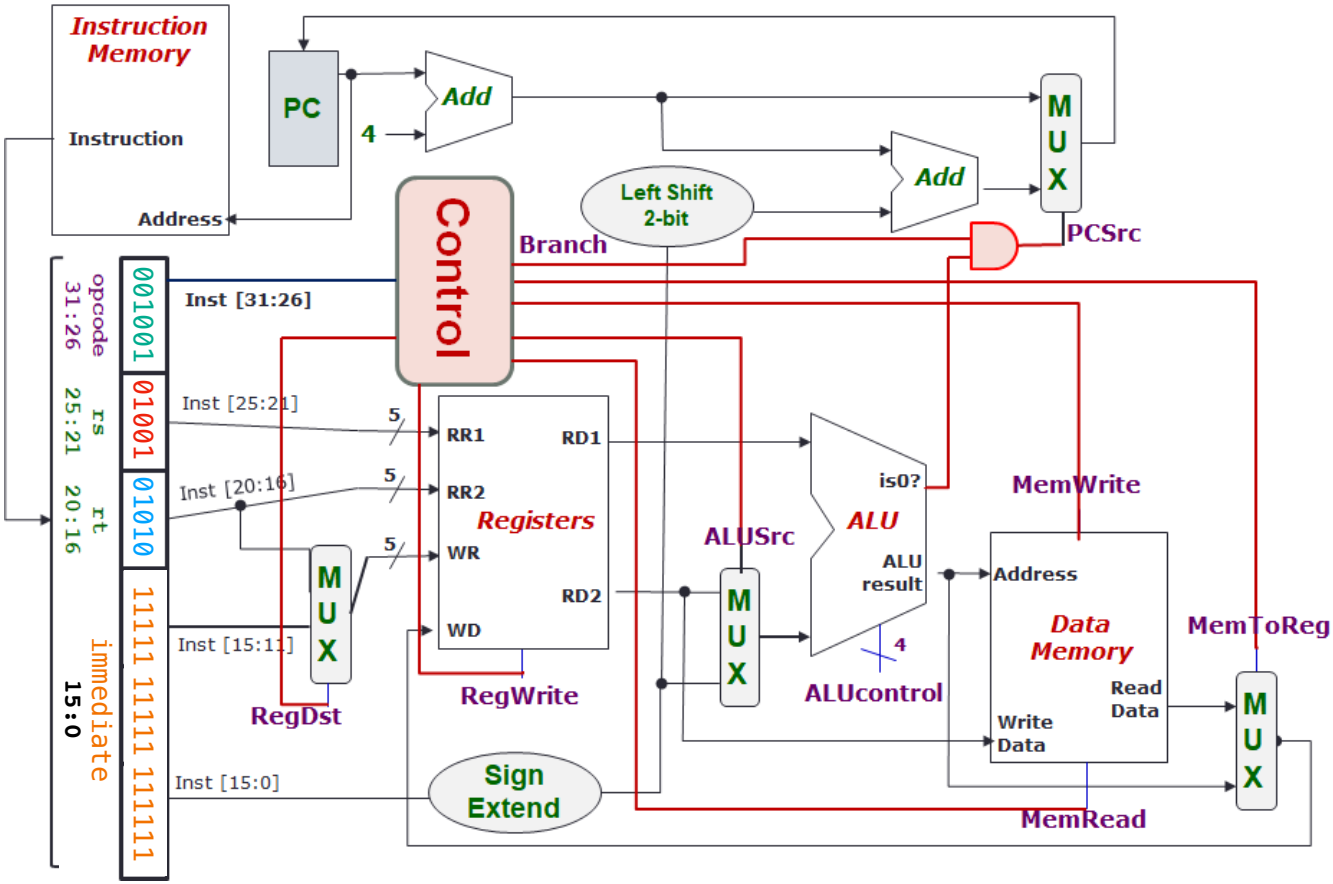


Walkthrough of I-Type Datapath

Unit 9: I-Type vs. R-Type Registers Usages

4: Write-Back:

- The **ALU** result (the sum of `$t1` and the `immediate` value –1) is fed into the final multiplexer (**MUX** labeled **MemToReg**).
- Because the Control Unit set **MemToReg** = 0, this **MUX** selects the value from the **ALU** result (rather than data from memory).

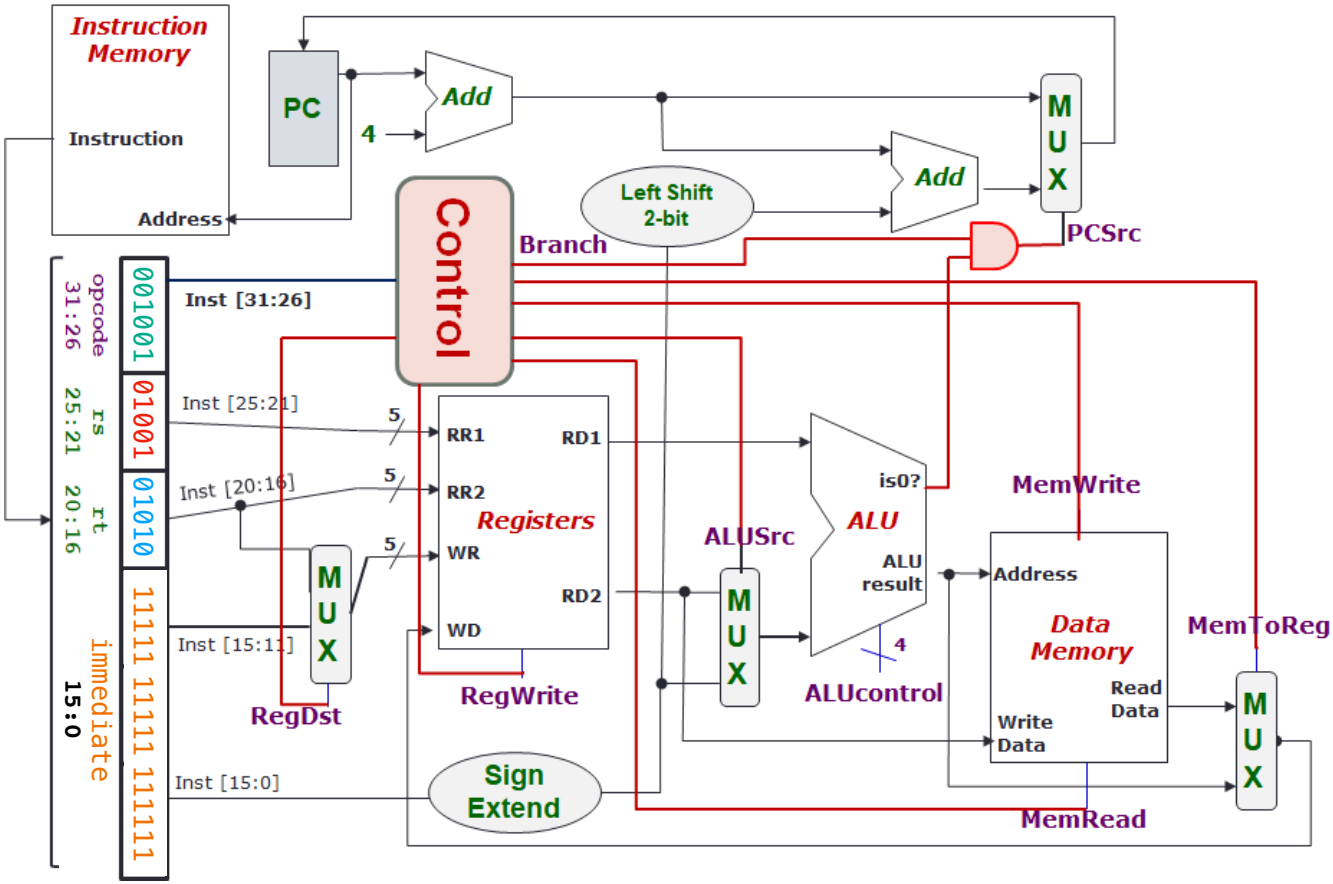


Walkthrough of I-Type Datapath

Unit 9: I-Type vs. R-Type Registers Usages

4: Write-Back:

- With **RegWrite** = 1 enabled, this final result is written back to the destination register specified by the `rt` field (`$t2`/register 10).

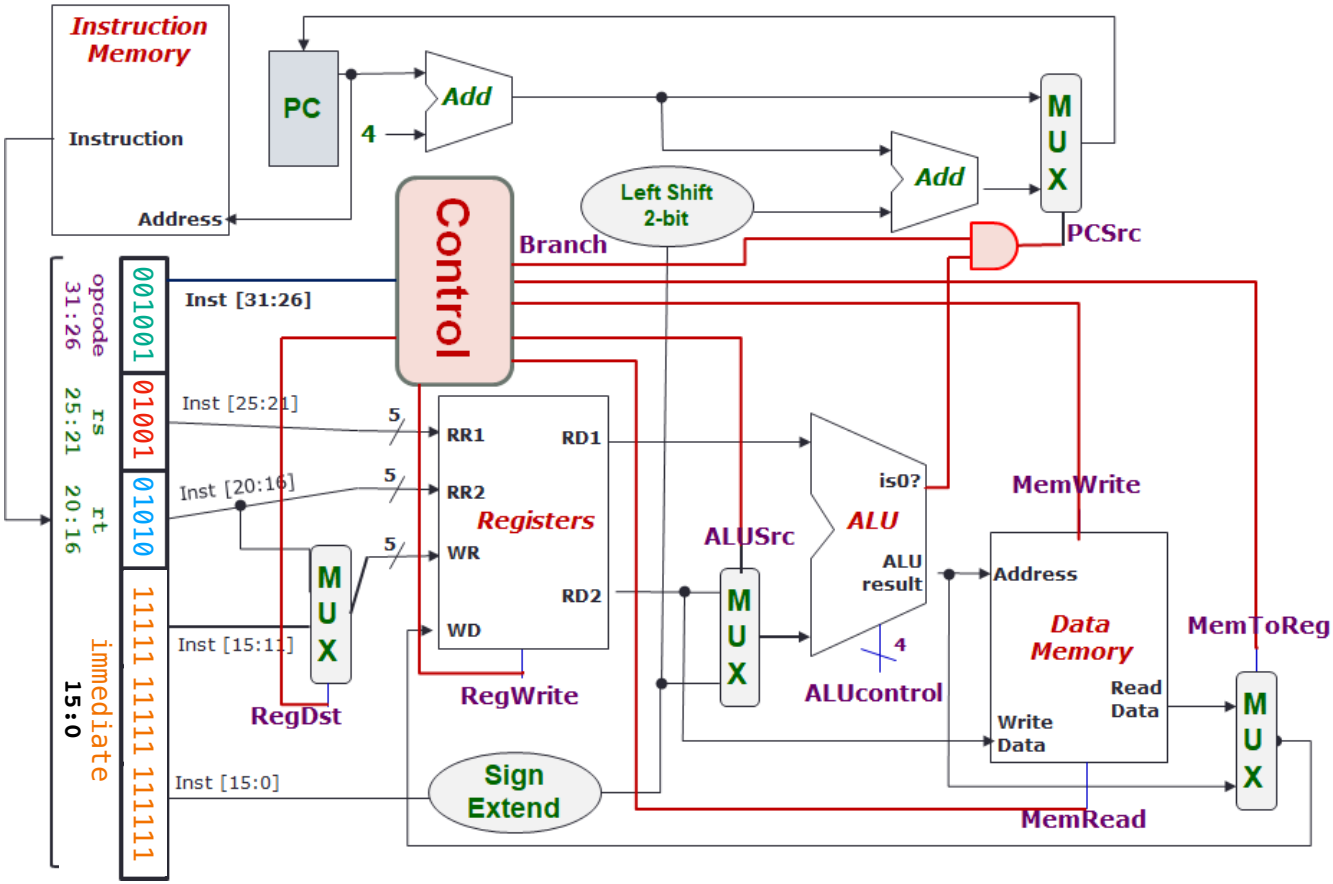


Walkthrough of I-Type Datapath

Unit 9: I-Type vs. R-Type Registers Usages

4: Write-Back:

- This selected value (the **ALU result**) is then fed back as **Write Data** (**WD**) to the **Register File**.
- On the next clock cycle, with **RegWrite** = 1, the value on **WD** is written into the register specified on the **Write Register** (**WR**) input, which is $t2 (register 10).

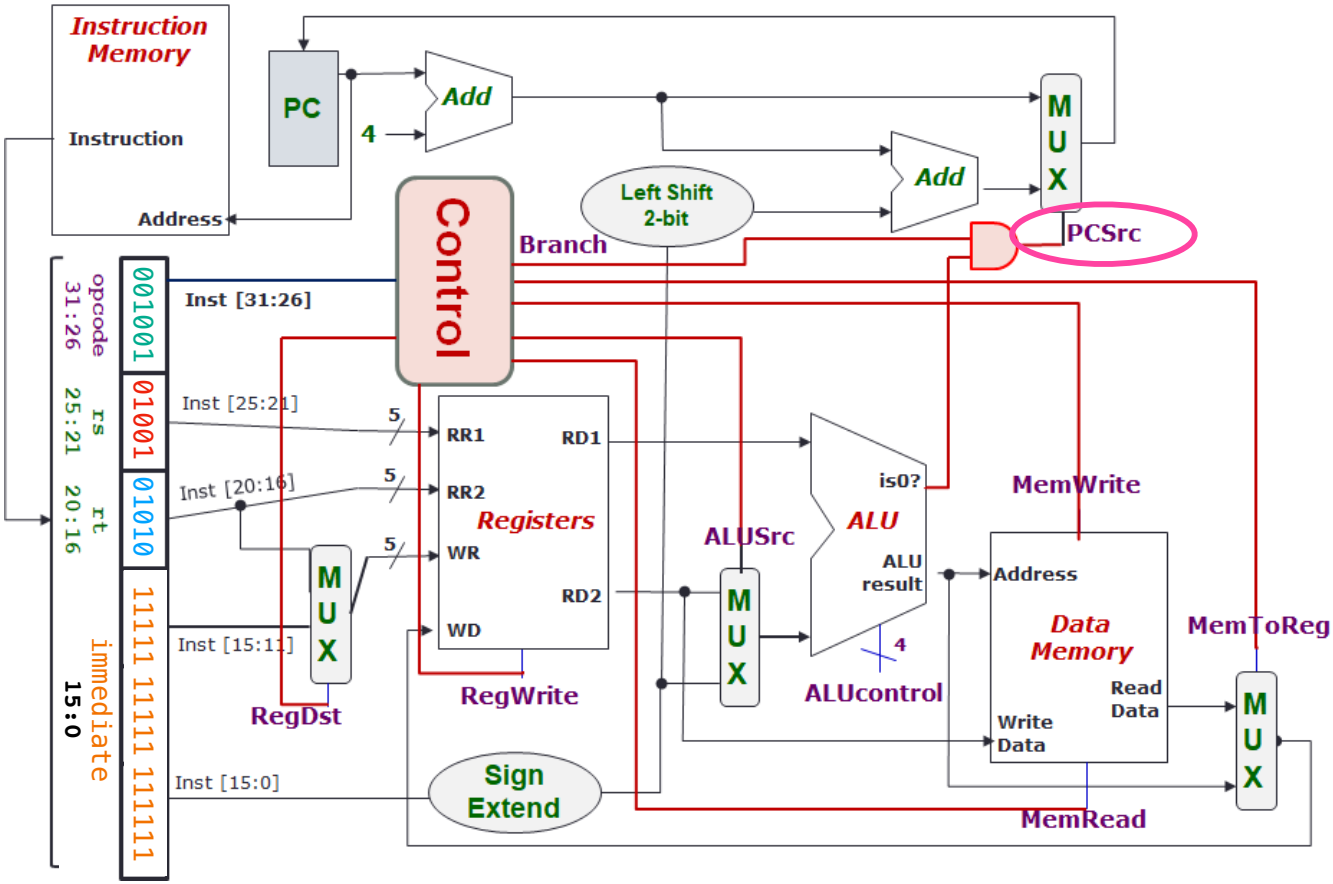


Walkthrough of I-Type Datapath

Unit 9: I-Type vs. R-Type Registers Usages

4: Write-Back:

- **PCSrc**
  - False (0): $PC' = $PC+4.
  - True (1): $PC' = ($PC+4)+ (immediate×4).
- **PCSrc** = 0 (False) because this is not a branch instruction
- The next instruction address is simply PC' = PC + 4.
- The PC proceeds sequentially to the next instruction.

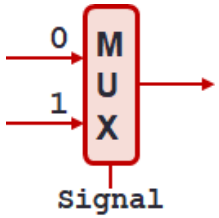


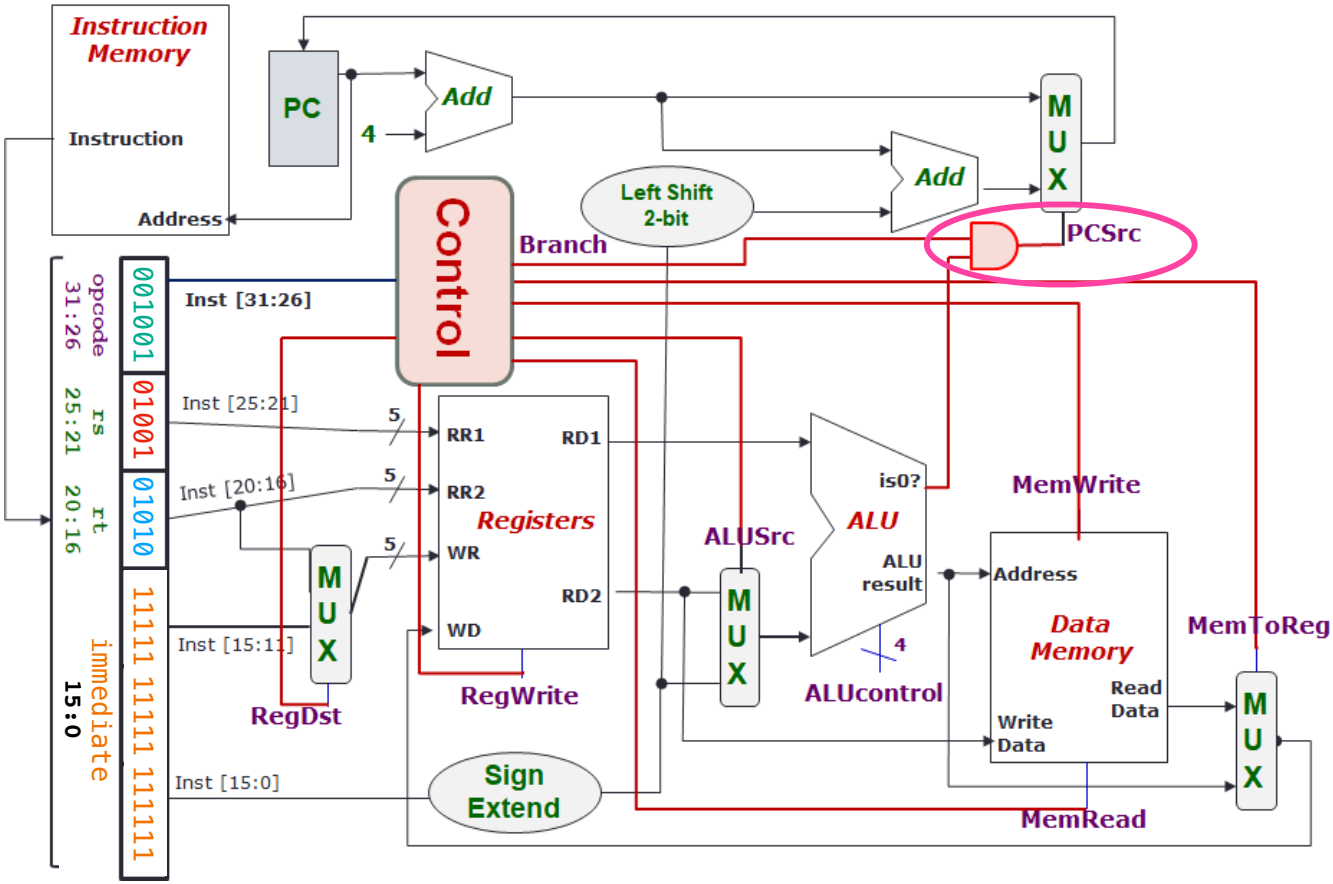
Walkthrough of I-Type Datapath

Unit 9: I-Type vs. R-Type Registers Usages

4: Write-Back:

- Branch
  - False (0): The operation is not a branch operation.
  - True (1): The operation is a branch operation (i.e., beq).
  - PCSrc = Branch & isZero? `addiu`: since **Branch** = 0, then **PCSrc** = 0 regardless of the **Zero** flag.



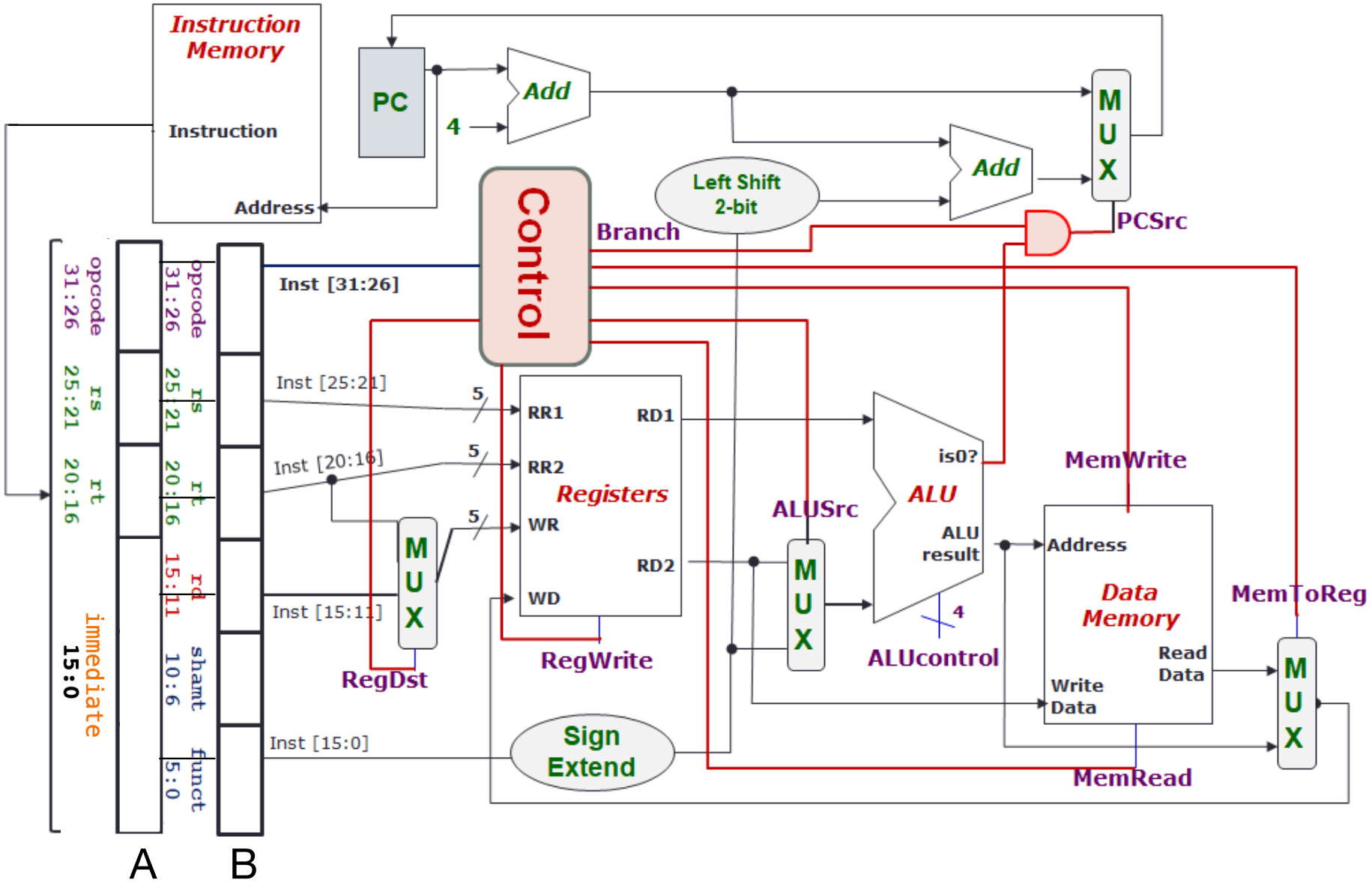


Critical Control Signals for `addiu`

Unit 9: I-Type vs. R-Type Registers Usages

Summary of Critical Control Signals for (**I-Type**) `addiu`

- **RegDst** = `0` (Write register comes from `rt` field)
- **ALUSrc** = 1 (ALU's second operand is the `immediate` value)
- **MemtoReg** = `0` (Value to write comes from the **ALU**, not **memory**)
- **RegWrite** = 1 (writing to a register)
- **MemRead** = `0` (not reading memory)
- **MemWrite** = `0` (not writing memory)
- **Branch** = `0` (This is not a branch instruction)


Instruction Memory
Instruction
Address
PC
4
Add
Add
Left Shift 2-bit
M U X
PCSrc
Branch
Control
Inst [31:26]
Inst [25:21]
Inst [20:16]
Inst [15:11]
Inst [15:0]
5
RR1
RR2
WR
WD
RD1
RD2
Registers
RegWrite
RegDst
ALUSrc
is0?
ALU
ALU result
4
ALUcontrol
MemWrite
Address
Data Memory
Read Data
Write Data
MemRead
MemToReg
Sign Extend
opcode 31:26
rs 25:21
rt 20:16
rd 15:11
shamt 10:6
funct 5:0
immediate 15:0
A
B

Critical Control Signals for `addiu`

Unit 9: I-Type vs. R-Type Registers Usages

Summary of Critical Control Signals for (**R-Type**) `add`

- **RegDst** = _____
- **ALUSrc** = _____
- **MemtoReg** = _____
- **RegWrite** = _____
- **MemRead** = _____
- **MemWrite** = _____
- **Branch** = _____



32-bit Constants

Unit 9: I-Type vs. R-Type Registers Usages

- I-Type instructions can have only 16-bit constants



- What if we want to load a 32-bit constant into a register?
- Can't have a 32-bit constant in I-Type instructions.
  - The sizes of all instructions are fixed to 32 bits
- Solution: use two instructions instead of one.
- Suppose we want: `$t1 = 0xAC5165D9` (32-bit constant)

`lui`: load upper immediate

```
lui $t1, 0xAC51
ori $t1, $t1, 0x65D9
```

	Upper 16 bits	Lower 16 bits
`# $t1`	`0xAC51`	`0x0000`
`# $t1`	`0xAC51`	`0x65D9`

32-bit Constants

Unit 9: I-Type vs. R-Type Registers Usages

- The Solution: Two Instructions (**`lui`** + **`ori`**)



- To load a 32-bit number, we use two separate instructions:
  - One to set the upper 16 bits of the target register.
  - One to set the lower 16 bits of the target register.

`lui`: load upper immediate

```
lui $t1, 0xAC51
ori $t1, $t1, 0x65D9
```

	Upper 16 bits	Lower 16 bits
`# $t1`	`0xAC51`	`0x0000`
`# $t1`	`0xAC51`	`0x65D9`

32-bit Constants

Unit 9: I-Type vs. R-Type Registers Usages

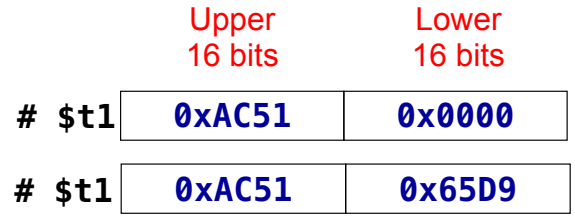
- The Solution: Two Instructions (**`lui`** + **`ori`**)



- The **`lui`** (Load Upper Immediate) Instruction
- Purpose: **`lui`** `$rt`, immediate loads a 16-bit immediate value into the upper 16 bits of the target register ($rt).
  - It then sets the lower 16 bits to zero.
  - The upper half (AC51) is in place, and the lower half is zeros (`0000`).

`lui`: load upper immediate

```
lui $t1, 0xAC51
ori $t1, $t1, 0x65D9
```



32-bit Constants

Unit 9: I-Type vs. R-Type Registers Usages

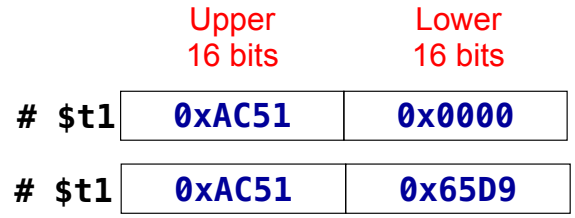
- The Solution: Two Instructions (**`lui`** + **`ori`**)



- Completing the Load with **`ori`** (Or Immediate)
- The **`lui`** instruction zeroed out the lower 16 bits.
  - We now need to put our desired lower 16 bits (`0x65D9`) into the register without disturbing the upper bits we just set.

`lui`: load upper immediate

```
lui $t1, 0xAC51
ori $t1, $t1, 0x65D9
```



32-bit Constants

Unit 9: I-Type vs. R-Type Registers Usages

- The Solution: Two Instructions (**`lui`** + **`ori`**)



- We use the **`ori`** (Or Immediate) instruction for this.
  - The logical OR operation is perfect:
    `0xAC510000 OR 0x000065D9 = 0xAC5165D9`
- The instruction would be: `ori $t1, $t1, 0x65D9`

`lui`: load upper immediate

```
lui $t1, 0xAC51
ori $t1, $t1, 0x65D9
```

	Upper 16 bits	Lower 16 bits
`# $t1`	`0xAC51`	`0x0000`
`# $t1`	`0xAC51`	`0x65D9`

32-bit Constants

Unit 9: I-Type vs. R-Type Registers Usages

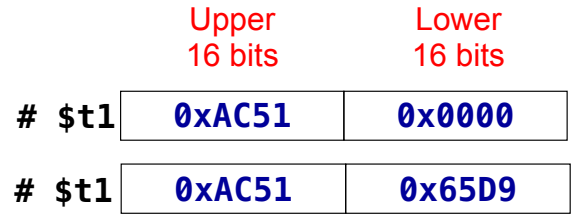
- The Solution: Two Instructions (**`lui`** + **`ori`**)



- This takes the current value of `$t1` (`0xAC510000`), ORs it with `0x65D9`, and stores the result (`0xAC5165D9`) back into `$t1`.

`lui`: load upper immediate

```
lui $t1, 0xAC51
ori $t1, $t1, 0x65D9
```



Pseudo Instructions

Unit 9: I-Type vs. R-Type Registers Usa

- Introduced by the assembler as if they were real instructions.
- Facilitate assembly language programming.

Pseudo-Instruction	Equivalent MIPS Instruction
`move $t1, $t2`	`addu $t1, $t2, $zero`
`not $t1, $t2`	`nor $t1, $t2, $zero`
`neg $t1, $t2`	`sub $t1, $zero, $t2`
`li $t1, -5`	`addiu $t1, $zero, -5`
`li $t1, 0xabcd1234`	`lui $t1, 0xabcd`
	`ori $t1, $t1, 0x1234`

- The MARS tool has a long list of pseudo-instructions.

MIPS Instruction Types

Type	-31- format (bits) -0-					
R	opcode (6)	rs (5)	rt (5)	rd (5)	shamt (5)	funct (6)
I	opcode (6)	rs (5)	rt (5)	immediate (16)		
J	opcode (6)	address (26)				

R-Type Instructions (Opcode 000000)

Main processor instructions that do not require a target address, immediate value, or branch displacement use an R-type coding format. This format has fields for specifying of up to three registers and a shift amount.

For instructions that do not use all of these fields, the unused fields are coded with all 0 bits. All R-type instructions use a 000000 opcode. The operation is specified by the function field.

opcode (6)	rs (5)	rt (5)	rd (5)	sa (5)	function (6)

Instruction		Function
add	rd, rs, rt	100000
addu	rd, rs, rt	100001
and	rd, rs, rt	100100
break		001101
div	rs, rt	011010
divu	rs, rt	011011
jalr	rd, rs	001001
jr	rs	001000
mfhi	rd	010000
mflo	rd	010010
mthi	rs	010001
mtlo	rs	010011
mult	rs, rt	011000
multu	rs, rt	011001
nor	rd, rs, rt	100111
or	rd, rs, rt	100101
sll	rd, rt, sa	000000
sllv	rd, rt, rs	000100
slt	rd, rs, rt	101010
sltu	rd, rs, rt	101011
sra	rd, rt, sa	000011
srav	rd, rt, rs	000111
srl	rd, rt, sa	000010
srlv	rd, rt, rs	000110
sub	rd, rs, rt	100010
subu	rd, rs, rt	100011
syscall		001100
xor	rd, rs, rt	100110

I-Type Instructions (All opcodes except 000000, 00001x, and 0100xx)

I-type instructions have a 16-bit immediate field that codes an immediate operand, a branch target offset, or a displacement for a memory operand. For a branch target offset, the immediate field contains the signed difference between the address of the following instruction and the target label, with the two low order bits dropped. The dropped bits are always 0 since instructions are word-aligned.

For the `bgez`, `bgtz`, `blez`, and `bltz` instructions, the rt field is used as an extension of the opcode field.

opcode (6)	rs (5)	rt (5)	immediate (16)

Instruction		Opcode	Notes
addi	rt, rs, immediate	001000	
addiu	rt, rs, immediate	001001	
andi	rt, rs, immediate	001100	
beq	rs, rt, label	000100	
bgez	rs, label	000001	rt = 00001
bgtz	rs, label	000111	rt = 00000
blez	rs, label	000110	rt = 00000
bltz	rs, label	000001	rt = 00000
bne	rs, rt, label	000101	
lb	rt, immediate(rs)	100000	
lbu	rt, immediate(rs)	100100	
lh	rt, immediate(rs)	100001	
lhu	rt, immediate(rs)	100101	
lui	rt, immediate	001111	
lw	rt, immediate(rs)	100011	
lwc1	rt, immediate(rs)	110001	
ori	rt, rs, immediate	001101	
sb	rt, immediate(rs)	101000	
slti	rt, rs, immediate	001010	
sltiu	rt, rs, immediate	001011	
sh	rt, immediate(rs)	101001	
sw	rt, immediate(rs)	101011	
swc1	rt, immediate(rs)	111001	
xori	rt, rs, immediate	001110	

Pseudo Instructions

Unit 9: I-Type vs. R-Type Registers Usa

What are Pseudo-Instructions?

- They are instructions that do not exist in the actual MIPS hardware instruction set.
- They are provided by the assembler (the program that converts text code into machine code) as a convenience for the programmer.
- The assembler sees a pseudo-instruction and automatically replaces it with one or more real, valid MIPS instructions.

MIPS Instruction Types

Type	-31- format (bits) -0-					
R	opcode (6)	rs (5)	rt (5)	rd (5)	shamt (5)	funct (6)
I	opcode (6)	rs (5)	rt (5)	immediate (16)		
J	opcode (6)	address (26)				

R-Type Instructions (Opcode 000000)

Main processor instructions that do not require a target address, immediate value, or branch displacement use an R-type coding format. This format has fields for specifying of up to three registers and a shift amount.

For instructions that do not use all of these fields, the unused fields are coded with all 0 bits. All R-type instructions use a 000000 opcode. The operation is specified by the function field.

opcode (6)	rs (5)	rt (5)	rd (5)	sa (5)	function (6)

Instruction		Function
add	rd, rs, rt	100000
addu	rd, rs, rt	100001
and	rd, rs, rt	100100
break		001101
div	rs, rt	011010
divu	rs, rt	011011
jalr	rd, rs	001001
jr	rs	001000
mfhi	rd	010000
mflo	rd	010010
mthi	rs	010001
mtlo	rs	010011
mult	rs, rt	011000
multu	rs, rt	011001
nor	rd, rs, rt	100111
or	rd, rs, rt	100101
sll	rd, rt, sa	000000
sllv	rd, rt, rs	000100
slt	rd, rs, rt	101010
sltu	rd, rs, rt	101011
sra	rd, rt, sa	000011
srav	rd, rt, rs	000111
srl	rd, rt, sa	000010
srlv	rd, rt, rs	000110
sub	rd, rs, rt	100010
subu	rd, rs, rt	100011
syscall		001100
xor	rd, rs, rt	100110

I-Type Instructions (All opcodes except 000000, 00001x, and 0100xx)

I-type instructions have a 16-bit immediate field that codes an immediate operand, a branch target offset, or a displacement for a memory operand. For a branch target offset, the immediate field contains the signed difference between the address of the following instruction and the target label, with the two low order bits dropped. The dropped bits are always 0 since instructions are word-aligned.

For the `bgez`, `bgtz`, `blez`, and `bltz` instructions, the rt field is used as an extension of the opcode field.

opcode (6)	rs (5)	rt (5)	immediate (16)

Instruction		Opcode	Notes
addi	rt, rs, immediate	001000	
addiu	rt, rs, immediate	001001	
andi	rt, rs, immediate	001100	
beq	rs, rt, label	000100	
bgez	rs, label	000001	rt = 00001
bgtz	rs, label	000111	rt = 00000
blez	rs, label	000110	rt = 00000
bltz	rs, label	000001	rt = 00000
bne	rs, rt, label	000101	
lb	rt, immediate(rs)	100000	
lbu	rt, immediate(rs)	100100	
lh	rt, immediate(rs)	100001	
lhu	rt, immediate(rs)	100101	
lui	rt, immediate	001111	
lw	rt, immediate(rs)	100011	
lwc1	rt, immediate(rs)	110001	
ori	rt, rs, immediate	001101	
sb	rt, immediate(rs)	101000	
slti	rt, rs, immediate	001010	
sltiu	rt, rs, immediate	001011	
sh	rt, immediate(rs)	101001	
sw	rt, immediate(rs)	101011	
swc1	rt, immediate(rs)	111001	
xori	rt, rs, immediate	001110	

Pseudo Instructions

Unit 9: I-Type vs. R-Type Registers Usages

Why are they useful?

- They make code easier to read and write.
- They act as helpful shortcuts, saving the programmer from writing longer, more complex sequences of real instructions.

- So, for example, if I say **`move $t1, $t2`** ,
- What will be the machine code?





The move $t1, $t2 instruction in MIPS assembly is not a “real” or native instruction.

It is a pseudo-instruction.

This means the assembler (the program that converts your assembly code into machine code) recognizes it and replaces it with a real, native instruction that performs the same action.

For move $t1, $t2, the assembler replaces it with:
addu $t1, $t2, $zero

Pseudo Instructions

Unit 9: I-Type vs. R-Type Registers Usages

Examples from the Table: `move $t1, $t2`

- Meaning: Copy the value from register `$t2` into register `$t1`.
- Implementation: `addu $t1, $t2, $zero`
- How it works: It adds the value in `$t2` to the value in `$zero` (which is always 0) and stores the result in `$t1`.
    - This is effectively a copy operation.

Op	rs	$t4	$t6	sa	funct
000000	_____	_____	_____	_____	_____

Op	rs	Rt	immediate
_____	_____	_____	____ ____ ____ ____



Pseudo Instructions

Unit 9: I-Type vs. R-Type Registers Usages

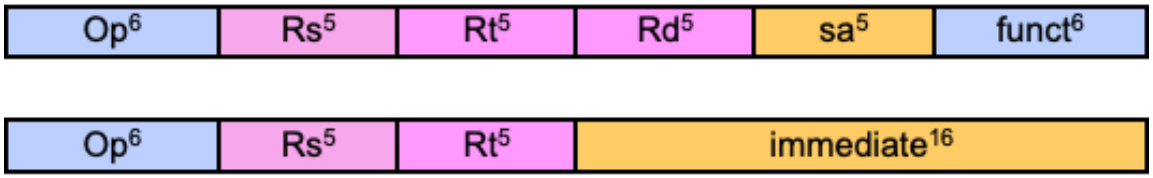
Examples from the Table: `not $t1, $t2`

- Meaning: Perform a bitwise logical NOT on the value in **`$t2`** (flip all bits) and store the result in **`$t1`**.
- Implementation: **`nor $t1, $t2, $zero`**
- How it works: NOR is “NOT OR”.
    - The operation A NOR `0` is equivalent to NOT A.
    - This is a clever way to achieve a NOT operation.

Pseudo Instructions

Unit 9: I-Type vs. R-Type Registers Usages

- If I say **`not  $t1, $t2`** ,
- What will be the machine code?



Op	rs	$t4	$t6	sa	funct
000000	_____	_____	_____	_____	_____

Op	rs	Rt	immediate
_____	_____	_____	____ ____ ____ ____



Pseudo Instructions

Unit 9: I-Type vs. R-Type Registers Usages

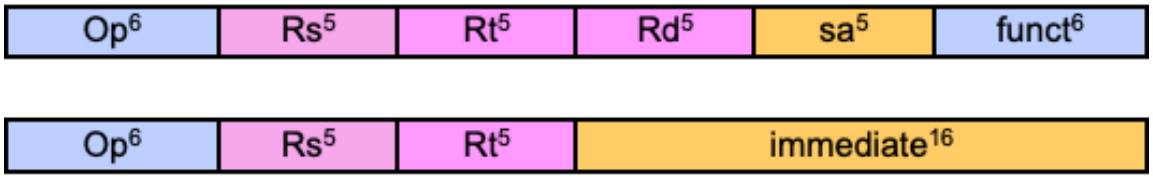
Examples from the Table: `neg $t1, $t2`

- Meaning: Negate the value in `$t2` (using two's complement) and store the result in `$t1`.
- Implementation: `sub $t1, $zero, $t2`
- How it works: It subtracts `$t2` from 0, which is the definition of arithmetic negation.

Pseudo Instructions

Unit 9: I-Type vs. R-Type Registers Usages

- If I say **`neg  $t1, $t2`** ,
- What will be the machine code?



Op	rs	$t4	$t6	sa	funct
000000	_____	_____	_____	_____	_____

Op	rs	Rt	immediate
_____	_____	_____	____ ____ ____ ____



Pseudo Instructions

Unit 9: I-Type vs. R-Type Registers Usages

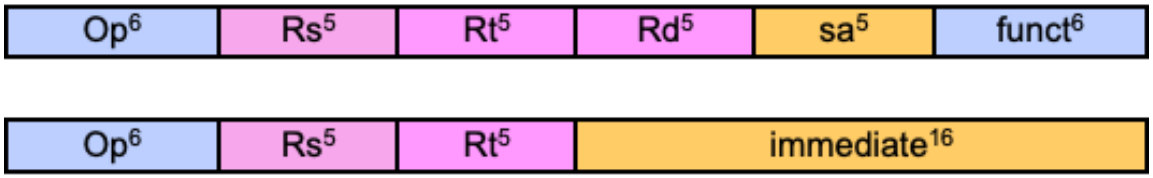
Examples from the Table: `li $t1, -5`

- Load Immediate
- Meaning: Load the small constant -5 into register **`$t1`**.
- Implementation: **`addiu $t1, $zero, -5`**
- How it works: It adds the sign-extended 16-bit value -5 to **`$zero`** and stores it in **`$t1`**.

Pseudo Instructions

Unit 9: I-Type vs. R-Type Registers Usages

- If I say `li  $t1, -5`,
- What will be the machine code?



Op	rs	$t4	$t6	sa	funct
000000	_____	_____	_____	_____	_____

Op	rs	Rt	immediate
_____	_____	_____	____ ____ ____ ____



Pseudo Instructions

Unit 9: I-Type vs. R-Type Registers Usages

Examples from the Table: `li $t1, 0xabcd1234`

- Load Immediate with a 32-bit constant
- This is the direct application of the concept for a 32-bit constant.
- Meaning: Load the large 32-bit constant into `$t1`.
- Implementation:

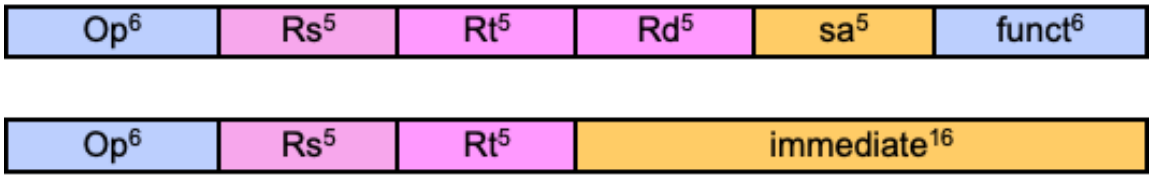
```
lui $t1, 0xAC51
ori $t1, $t1, 0x65D9
```

	Upper 16 bits	Lower 16 bits
# $t1	0xAC51	0x0000
# $t1	0xAC51	0x65D9

Pseudo Instructions

Unit 9: I-Type vs. R-Type Registers Usages

- If I say `li $t1, 0xabcd1234`,
- What will be the machine code?



Op	rs	$t4	$t6	sa	funct
000000	_____	_____	_____	_____	_____

Op	rs	Rt	immediate
_____	_____	_____	____ ____ ____ ____

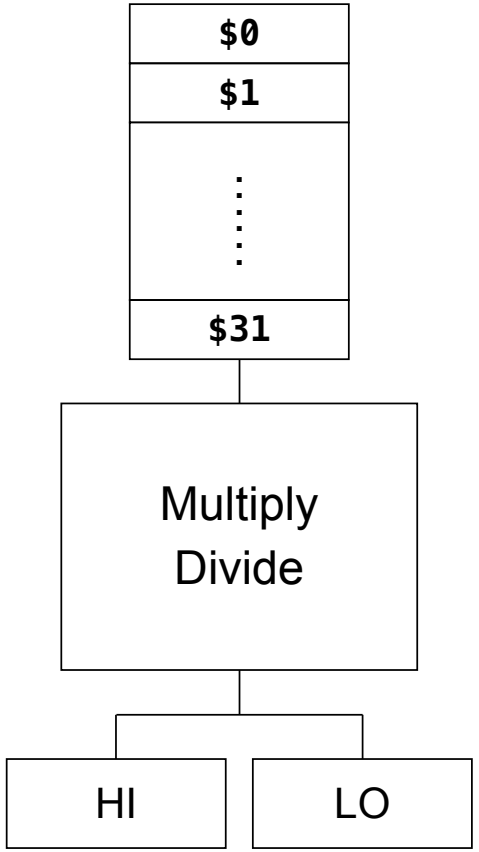


Integer Multiplication & Division in MIPS

Integer Multiplication in MIPS

Unit 9: I-Type vs. R-Type Registers Usages

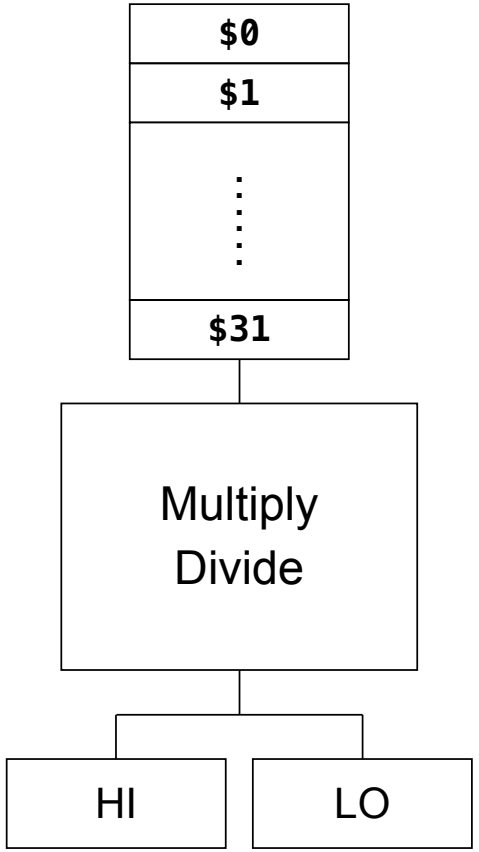
- Multiply instructions
  - `mult  $s1,$s2` #Signed multiplication
  - `multu $s1,$s2` #Unsigned multiplication
- 32-bit multiplication produces a 64-bit Product.
- Separate pair of 32-bit registers
  - HI = high-order 32-bit of product.
  - LO = low-order 32-bit of product.
- MIPS also has a special mul instruction
  - `mul  $s0,$s1,$s2` # $s0 = $s1 × $s2
  - Put the low-order 32 bits into the destination register.
- HI & LO are undefined.



Integer Division in MIPS

Unit 9: I-Type vs. R-Type Registers Usages

- Divide instructions
  - `div  $s1,$s2` #Signed division
  - `divu $s1,$s2` #Unsigned division
- Division produces a quotient and a remainder.
- Separate pair of 32-bit registers
  - HI = 32-bit remainder
  - LO = 32-bit quotient
  - If the divisor is `0`, then the result is unpredictable.
- Moving data from HI/LO to MIPS registers.
  - `mfhi Rd` #move from HI to Rd
  - `mflo Rd` #move from LO to Rd



Multiplication & Division Instructions

Unit 9: I-Type vs. R-Type Registers Usages

Instruction	Meaning	Op	Rs	Rt	Rd	sa	func
`mult Rs, Rt`	`Hi, Lo = Rs × Rt`	0	Rs	Rt	0	0	0x18
`multu Rs, Rt`	`Hi, Lo = Rs × Rt`	0	Rs	Rt	0	0	0x19
`mul Rd, Rs, Rt`	`Rd = Rs × Rt`	1C	Rs	Rt	Rd	0	0x02
`div Rs, Rt`	`Hi, Lo = Rs / Rt`	0	Rs	Rt	0	0	0x1A
`divu Rs, Rt`	`Hi, Lo = Rs / Rt`	0	Rs	Rt	0	0	0x1B
`mfhi Rd`	`Rd = Hi`	0	0	0	Rd	0	0x10
`mflo Rd`	`Rd = Lo`	0	0	0	Rd	0	0x12

- Signed arithmetic: **`mult`**, **`div`** (**`Rs`** and **`Rt`** are signed).
  - **`Lo`** = 32-bit low-order and **`Hi`** = 32-bit high-order of multiplication.
  - **`Lo`** = 32-bit quotient and **`Hi`** = 32-bit remainder of division.
- Unsigned arithmetic: **`multu`**, **`divu`** (**`Rs`** and **`Rt`** are unsigned).
- NO arithmetic exception can occur.

Multiplication & Division Instructions

Unit 9: I-Type vs. R-Type Registers Usages

Basic	Source		
addiu $8,$0,0x00000005	6:	li $t0, 5	# $t0 = 5
addiu $9,$0,0x00000003	7:	li $t1, 3	# $t1 = 3
mult $8,$9	10:	mult $t0, $t1	# 5 × 3 = 15
mflo $10	13:	mflo $t2	# $t2 = 15 (low 32 bits)
mfhi $11	14:	mfhi $t3	# $t3 = 0 (high 32 bits – zero for small numbers)
addiu $2,$0,0x00000001	17:	li $v0, 1	# system call for print integer
addu $4,$0,$10	18:	move $a0, $t2	# load result into argument register
syscall	19:	syscall	
addiu $2,$0,0x0000000a	22:	li $v0, 10	
syscall	23:	syscall	

Control Flow

Control Flow: Branch and Jump Instruction

Unit 9: I-Type vs. R-Type Registers Usages

- High-level programming languages provide constructs:
  - To make decisions in a program: IF-ELSE
  - To repeat the execution of a sequence of instructions: LOOP
- The ability to make decisions and repeat a sequence of instructions distinguishes a computer from a calculator.
- All computer architectures provide control flow instructions.
- Essential for making decisions and repetitions.
- These are the conditional branch and jump instructions.

Control Flow: Branch and Jump Instruction

Unit 9: I-Type vs. R-Type Registers Usages

- MIPS compare and branch instructions:

```
beq Rs, Rt, label if (Rs == Rt) branch to label
bne Rs, Rt, label if (Rs != Rt) branch to label
```

- MIPS compares to zero & branch instructions:
  - Compared to zero is used frequently and implemented efficiently

```
bltz Rs, label  if (Rs < 0) branch to label
bgtz Rs, label  if (Rs > 0) branch to label
blez Rs, label  if (Rs <= 0) branch to label
bgez Rs, label  if (Rs >= 0) branch to label
```

- **`beqz`** and **`bnez`** are defined as pseudo-instructions.

Control Flow: Branch and Jump Instruction

Unit 9: I-Type vs. R-Type Registers Usages

- Branch Instructions are of the I-type Format:



Instruction	Meaning	Op	Rs	Rt	Immediate
`beq $t0, $t1, loop`	`if $t0 == $t1, jump to loop`	0x4	$t0	$t1	16-bit Offset
`bne $s0, $0, process`	`if $s0 != 0, jump to process` `if $s0 == 0, continue sequentially`	0x5	$s0	$0	16-bit Offset
`blez $t2, negative_case`	`if $t2 <= 0, jump to negative_case` `if $t2 > 0, continue`	0x6	$t2	0	16-bit Offset
`bgtz $a0, positive`	`if $a0 > 0, jump to positive` `if $a0 <= 0, continue`	0x7	$a0	0	16-bit Offset
`bltz $t5, handle_negative`	`if $t5 < 0, jump to handle_negative` `if $t5 >= 0, continue`	0x1	$t5	0	16-bit Offset
`bgez $s1, non_negative`	`if $s1 >= 0, jump to non_negative` `if $s1 < 0, continue`	0xf	$s1	1	16-bit Offset

Control Flow: Branch and Jump Instru

Unit 9: I-Type vs. R-Type Registers Usages





- Branch Instructions are of the I-type Format:



Instruction	Meaning	Op	Rs	Rt	Immediate
`beq $t0, $t1, loop`	`if $t0 == $t1, jump to loop`	0x4	$t0	$t1	16-bit Offset
`bne $s0, $0, process`	`if $s0 != 0, jump to process` `if $s0 == 0, continue sequentially`	0x5	$s0	$0	16-bit Offset
`blez $t2, negative_case`	`if $t2 <= 0, jump to negative_case` `if $t2 > 0, continue`	0x6	$t2	0	16-bit Offset
`bgtz $a0, positive`	`if $a0 > 0, jump to positive` `if $a0 <= 0, continue`	0x7	$a0	0	16-bit Offset
`bltz $t5, handle_negative`	`if $t5 < 0, jump to handle_negative` `if $t5 >= 0, continue`	0x1	$t5	0	16-bit Offset
`bgez $s1, non_negative`	`if $s1 >= 0, jump to non_negative` `if $s1 < 0, continue`	0xf	$s1	1	16-bit Offset

- What are the machine codes?

PC Calculation

Unit 9: I-Type vs. R-Type Registers Usages

- The branch instructions modify the PC register only
- PC-Relative addressing:
  - If (branch is taken) PC = PC + 4 + 4×offset else PC = PC+4

    Current PC = `0x00400000` (instruction address)
    Offset = 3 (from assembly)
    The branch is taken
  - Calculation:
    - PC = PC + 4 + (4 × offset)
    - PC = `0x00400000` + 4 + (4 × 3)
    - PC = `0x00400000` + 4 + 12
    - PC = `0x00400000` + 16
    - PC = `0x00400010`

PC Calculation Example

Unit 9: I-Type vs. R-Type Registers Usages

- `bgtz $a0, kitti`
- Assembly Code:

```
0x00400000: addi $a0, $0, 5        # $a0 = 5
0x00400004: addi $t0, $0, 10       # $t0 = 10
0x00400008: bgtz $a0, kitti        # Branch if $a0 > 0
0x0040000C: addi $t1, $0, 1        # This gets SKIPPED
0x00400010: j end                  # This gets SKIPPED
0x00400014: kitti:                 # Branch target label
0x00400014: addi $t1, $0, 100      # $t1 = 100
0x0040001C: end:
0x0040001C: addi $t2, $0, 999      # $t2 = 999
```

PC Calculation Example

Unit 9: I-Type vs. R-Type Registers Usages

- `bgtz $a0, positive`
- Step-by-Step Execution:
  - At PC = 0x00400008, the `bgtz` instruction executes
    - `$a0` = 5 (which is > 0), so `branch` is TAKEN
  - Calculate the offset to the `positive` label:
    - Current instruction: 0x00400008
    - Target instruction: 0x00400014
  - Instructions to jump: (0x00400014 - 0x00400008 - 4) / 4

    = (12 - 4) / 4

    = 8 / 4

    = 2 instructions

PC Calculation Example

Unit 9: I-Type vs. R-Type Registers Usages

- `bgtz $a0, positive`
- Assembly Code:

```
0x00400000: addi $a0, $0, -5        # $a0 = -5
0x00400004: addi $t0, $0, 10        # $t0 = 10
0x00400008: bgtz $a0, positive      # Branch if $a0 > 0
0x0040000C: addi $t1, $0, 1         # This gets SKIPPED
0x00400010: j end                   # This gets SKIPPED
0x00400014: positive:               # Branch target label
0x00400014: addi $t1, $0, 100       # $t1 = 100
0x00400018: end:
0x00400018: addi $t2, $0, 999       # $t2 = 999
```

PC Calculation Example

Unit 9: I-Type vs. R-Type Registers Usages

- `bgtz $a0, positive`
- PC Calculation:

  PC = PC + 4 + (4 × offset)
  PC = `0x00400008` + 4 + (4 × 2)
  PC = `0x00400008` + 4 + 8
  PC = `0x00400008` + 12
  PC = `0x00400014`

- Result: Next instruction executed is at 0x00400014 (the positive label)
- An **offset** in **branch** instructions means the number of instructions to jump forward or backward from the next instruction (PC+4).

PC Calculation

Unit 9: I-Type vs. R-Type Registers Usages

- Key Points:
  1. **Offset = 0 would jump to the very next instruction (PC+4)**
  2. **Offset = 1 would skip one instruction**
  3. **Offset = 2 would skip two instructions**
  4. **Negative offset jumps backward in the code**

Unconditional Jump Instruction

Unit 9: I-Type vs. R-Type Registers Usages

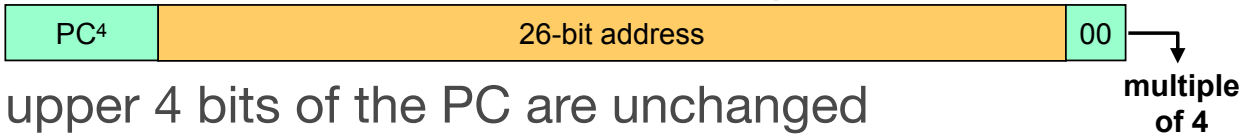
- The unconditional Jump instruction has the following syntax:

```
j   label  #jump to label
. . .
label:
```

- The jump instruction is always taken.
- The jump instruction is of the J-Type format:



- The jump instruction modifies the program counter PC:



- The upper 4 bits of the PC are unchanged

Interpreting an IF Statement

Unit 9: I-Type vs. R-Type Registers Usages

- Consider the following IF statement:

```
if (a == b)
    c = d + e; //addu  $t2, $t3, $t4
else
    c = d – e; //subu  $t2, $t3, $t4
```

  Given that a, b, c, d, e are in `$t0` … `$t4` respectively

- How to translate the above IF statement?

```
        bne   $t0, $t1, else
        addu  $t2, $t3, $t4
        j     next
else:   subu  $t2, $t3, $t4
next:   . . .
```

Interpreting an IF Statement

Unit 9: I-Type vs. R-Type Registers Usages

```
            bne   $t0, $t1, else
            addu  $t2, $t3, $t4
            j     next
      else: subu  $t2, $t3, $t4
      next: . . .
```

1. bne $t0, $t1, else # Test the condition
   - If a != b, jump to the `else` label
   - If a == b, continue to the `next` instruction
2. addu $t2, $t3, $t4 - THEN clause (only executed if a == b)
   - c = d + e

Interpreting an IF Statement

Unit 9: I-Type vs. R-Type Registers Usages

```
            bne   $t0, $t1, else
            addu  $t2, $t3, $t4
            j     next
      else: subu  $t2, $t3, $t4
      next: . . .
```

3. j next - jump over the `else` clause
    - Prevents executing the `else` part when we did the `next` part
4. else: subu $t2, $t3, $t4 - ELSE clause (only executed if a != b)
    - c = d - e
5. next: - Both paths reunite here

References

References

Reference Materials

- William Stallings, “Computer Organization and Architecture Designing for Performance,” Tenth Edition, *Pearson*, 2016
- Aiman El-Maleh, “Computer Organization: Lectures,” https://faculty.kfupm.edu.sa, accessed August 26, 2025
- CS2100: Computer Organization, https://www.comp.nus.edu.sg/~adi-yoga/CS2100/ch08c2/



